# Polytron Indonesia IC Design Engineer Internship Program

## Module *Hands-on*

From RTL to GDSII

TRAINEE ver. Day 3

## Authors

Ir. Agus Bejo, S.T., M.Eng., D.Eng., IPM.

Bobby Rian Dewangga, S.T., M.Eng.

Byan Cahaya Rahman Arvendy

POLYTRON

In collaboration with ICDEC

# MODUL V

# Modul *Hand-Ons* Digital IC Design
# Polytron Indonesia IC Design Engineer Internship
# Program MSIB *Batch* 7

Intermezzo

*Secure Shell (SSH) dan Secure File Transfer Protocol (SFTP)*

---

## A. *Overview*

*Secure Shell* (SSH) dan *Secure File Transfer Protocol* (SFTP) adalah dua protokol penting yang digunakan dalam dunia teknologi untuk mengakses, mengelola, dan mentransfer data dengan aman melalui jaringan. Modul ini bertujuan memberikan pemahaman dasar mengenai konsep, manfaat, dan penggunaan SSH serta SFTP dalam lingkungan komputasi modern.

Tujuan dari modul ini adalah:

1. Memahami konsep dasar dan manfaat SSH dan SFTP.
2. Menggunakan SSH untuk mengakses server jarak jauh secara aman.
3. Melakukan *transfer file* menggunakan SFTP dengan berbagai operasi dasar.

## B. *Secure Shell* (SSH)

SSH (*Secure Shell*) adalah protokol jaringan yang digunakan untuk mengakses dan mengelola server secara aman melalui koneksi terenkripsi. SSH menggantikan metode akses jarak jauh yang tidak aman seperti Telnet, dengan menyediakan autentikasi menggunakan *username-password* atau kunci publik/privat, serta melindungi komunikasi dari penyadapan. Protokol ini sangat penting untuk administrasi server jarak jauh secara aman. Berikut adalah perintah-perintah SSH yang umum digunakan:

1. Menghubungkan ke server

```
ssh username@server_address
```

2. Menggunakan *port* tertentu

```
ssh -p port username@server_address
```

3. Menjalankan *remote command*

```
ssh username@server_address [command]
```

4. Menggunakan X11 *Forwarding*

```
ssh -X username@server_address
```

** note :*

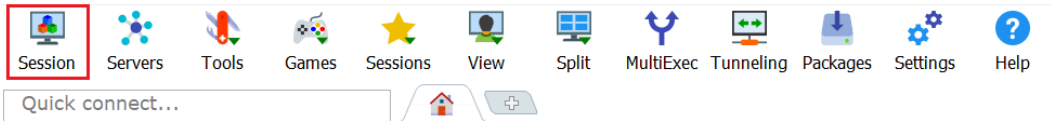
- *X11 Forwarding digunakan untuk mengoperasikan GUI dari aplikasi yang berjalan di remote server, sehingga tampilannya dapat diakses pada layar komputer lokal melalui koneksi SSH*
- *Fitur ini sudah otomatis diaktifkan oleh MobaXterm, sehingga tidak perlu tambahan flag -X saat login SSH dengan MobaXterm*

5. Keluar dari SSH

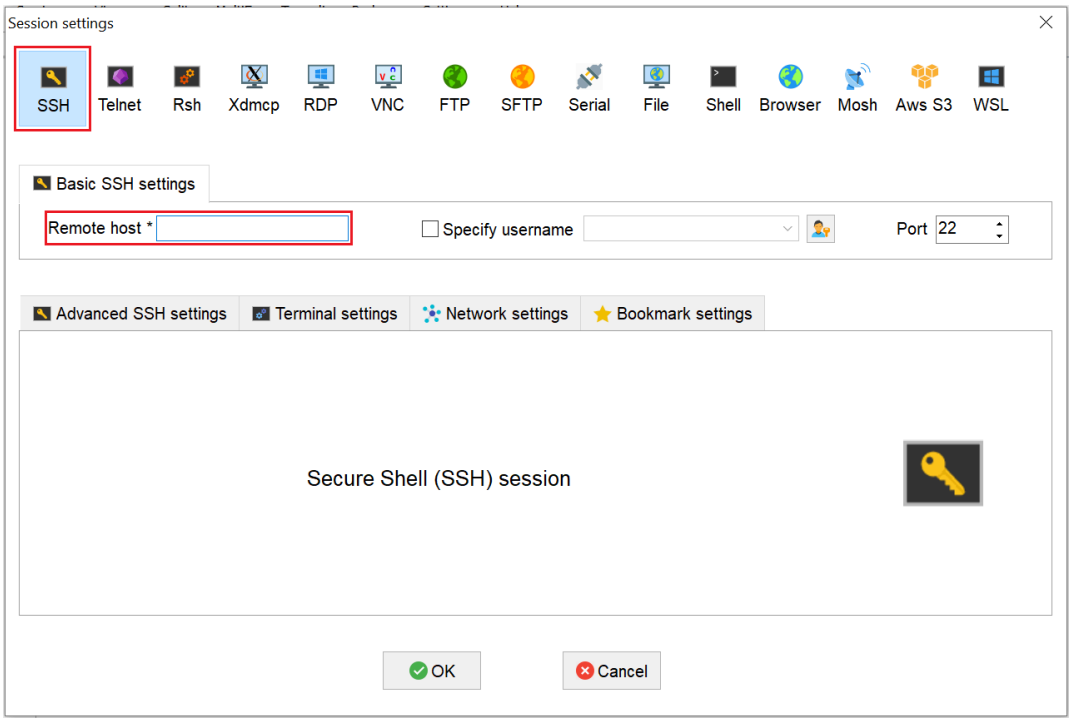
```
exit
```

## ● Melakukan *Login* SSH menggunakan MobaXterm

1. Pilih *Session*



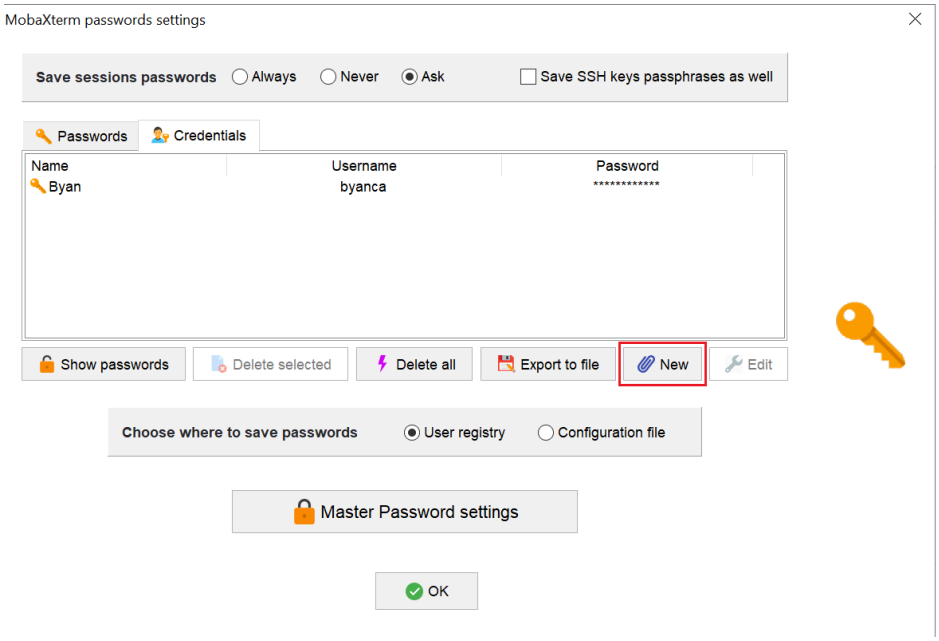

2. Pilih Bagian SSH. Kemudian masukkan *Remote Host*



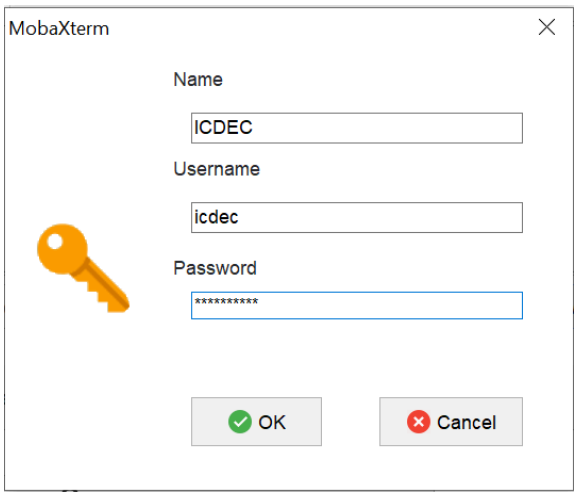
3. Centang kotak *Specify username*. Kemudian klik *icon* berikut




4. Kemudian pilih *New*




5. Kemudian masukkan *Name*, *Username*, dan *Password*. Jika sudah klik `OK`




6. Setelahnya klik `OK` pada `MobaXterm passwords settings`

7. Pilih *User* yang sesuai




8. Selanjutnya klik `OK`. Maka *login* SSH berhasil dilakukan dengan *user* tersebut.

## C. *Secure File Transfer Protocol* (SFTP)

SFTP (*Secure File Transfer Protocol*) adalah protokol berbasis SSH yang digunakan untuk mentransfer file secara aman antara klien dan server. Berbeda dengan *File Transfer Protocol* (FTP), SFTP mengenkripsi semua data, termasuk file dan kredensial, sehingga melindungi informasi sensitif. Dengan dukungan operasi seperti *upload*, *download*, dan pengelolaan file lainnya, SFTP menjadi solusi yang aman dan andal untuk kebutuhan transfer data. Berikut adalah *command* SFTP yang umum digunakan:

1. Mengakses SFTP

```
sftp username@server_address
```

2. Navigasi Direktori
    a. Pada Lokal
        - Melihat isi direktori

```
lls
```

        - Berpindah direktori

```
lcd path/to/directory
```

        - Menampilkan direktori saat ini

```
lpwd
```

b. Pada *Remote Server*

- Melihat isi direktori

```
ls
```

- Berpindah direktori

```
cd path/to/directory
```

- Menampilkan direktori saat ini

```
pwd
```

3. *Transfer File*

a. Mengunggah *File* ke Server

- *Upload* file

```
put local_file
```

- *Upload* direktori

```
put -r local_directory/
```

b. Mengunduh *File* dari Server

- *Download* file

```
get local_file
```

- *Download* direktori

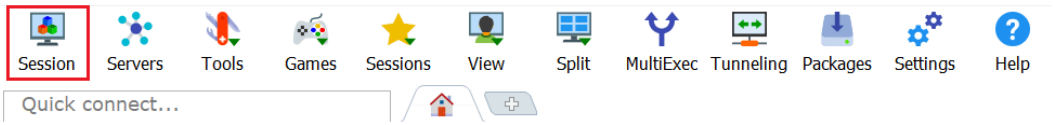
```
get -r local_directory/
```

4. Keluar dari SFTP

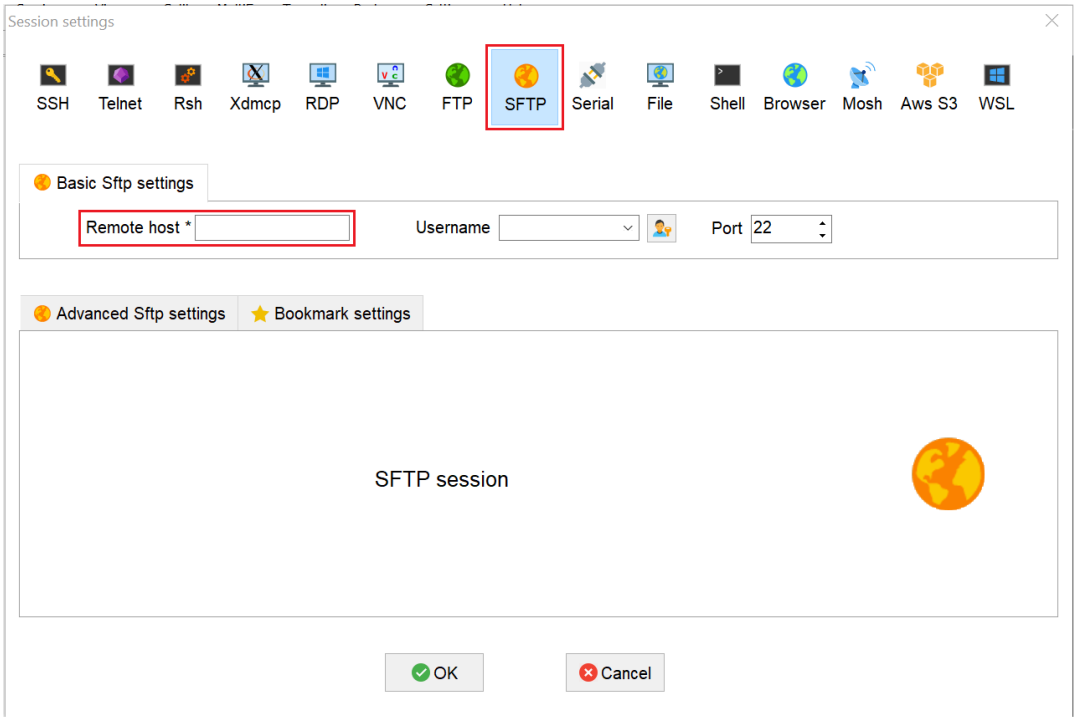
```
exit
```

## ● Melakukan *Login* SFTP menggunakan MobaXterm

1. Pilih *Session*



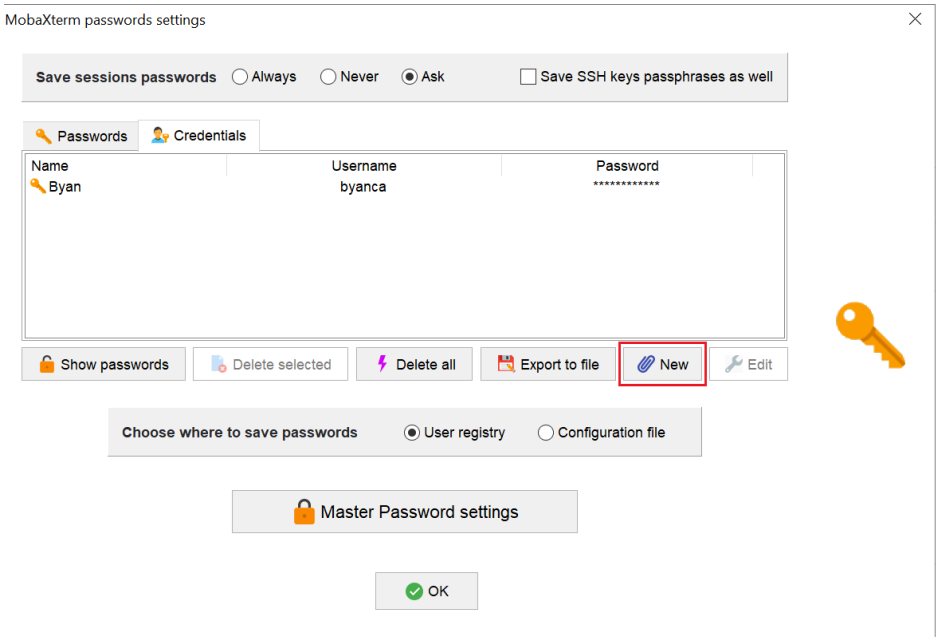
2. Pilih Bagian SFTP. Kemudian masukkan *Remote Host*



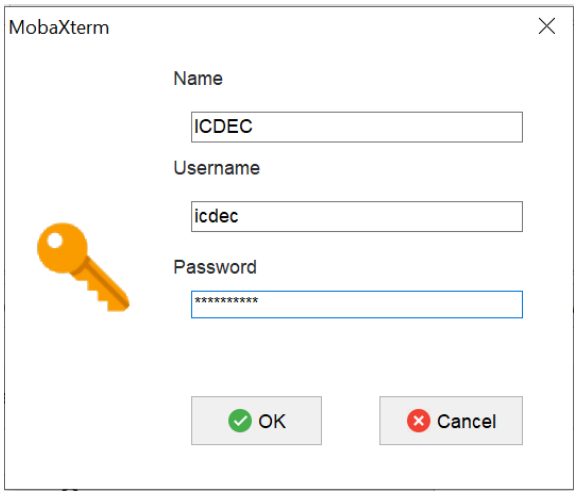

3. Centang kotak *Specify username*. Kemudian klik *icon* berikut



4. Kemudian pilih *New*




5. Kemudian masukkan *Name*, *Username*, dan *Password*. Jika sudah klik `OK`




6. Setelahnya klik `OK` pada `MobaXterm passwords settings`

7. Pilih *User* yang sesuai




8. Selanjutnya klik `OK`. Maka *login* SFTP berhasil dilakukan dengan *user* tersebut

# MODUL VI

# Modul *Hand-Ons* Digital IC Design
# Polytron Indonesia IC Design Engineer Internship
# Program MSIB *Batch* 7

*Verilog to GDSII File using Cadence*

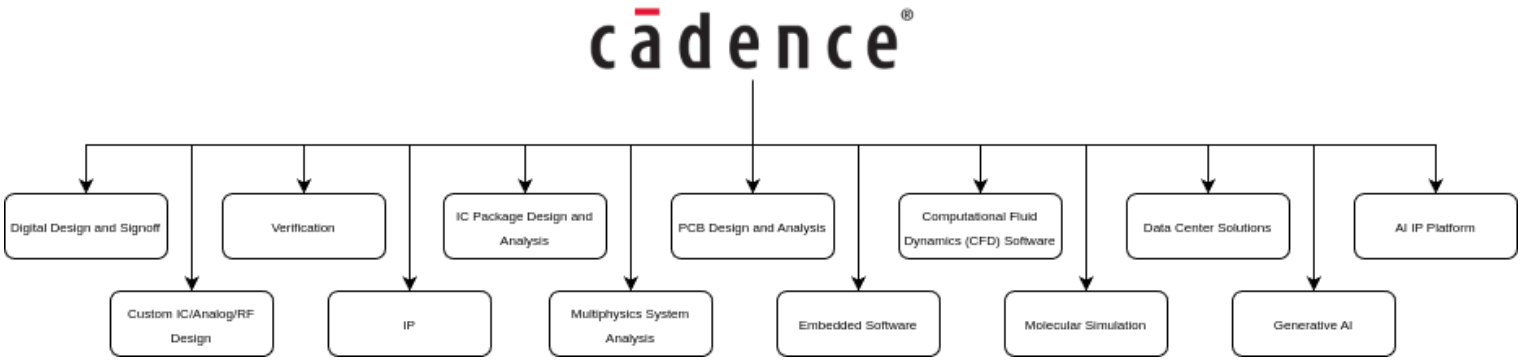
---

## A. Tujuan Pelatihan

Pelatihan ini bertujuan untuk memberikan pemahaman mendalam dalam desain IC digital, mulai dari tahap deskripsi RTL (*Register Transfer Level*) hingga menghasilkan layout fisik yang siap untuk manufaktur. Peserta akan belajar bagaimana menggunakan *software* Cadence untuk melakukan sintesis desain digital yang optimal dan memahami seluruh alur kerja dari Verilog ke GDSII. Beberapa hasil yang diharapkan dari pelatihan ini adalah sebagai berikut:

- Peserta memahami proses desain IC digital dari tahap RTL hingga layout fisik, mencakup semua langkah penting dalam alur desain IC.
- Peserta dapat mendokumentasikan dan memverifikasi desain digital agar sesuai dengan spesifikasi industri dan standar manufaktur.

## B. Pengenalan Singkat Cadence *Tools*




**Gambar 1. Produk *Software* Cadence berdasarkan Kategori**

Cadence menyediakan berbagai *tool* untuk mendukung desain dan analisis sistem elektronik. *Digital Design and Signoff* digunakan untuk desain *chip* digital dari logika hingga implementasi fisik, sementara *Custom IC/Analog/RF Design* berfokus pada desain IC analog, RF, dan *mixed-signal*. Verification memastikan validasi desain melalui simulasi dan formal verification, dan *IP* menyediakan *design block* siap pakai untuk mempercepat pengembangan. *IC Package Design and Analysis* serta *PCB Design and Analysis* mendukung desain dan analisis packaging IC serta PCB. Tool *Multiphysics System Analysis* digunakan untuk analisis termal, elektromagnetik, dan mekanikal. Cadence juga menawarkan solusi untuk *Embedded Software*, *Computational Fluid Dynamics (CFD) Software*, *Molecular Simulation*, dan *Data*

*Center Solutions*. Dengan *Generative AI* dan *AI IP Platform*, Cadence mengintegrasikan teknologi AI untuk otomatisasi dan akselerasi desain.

## C. *GDSII File*

GDSII (*Graphic Data System* II) adalah format file standar dalam industri desain IC yang digunakan untuk menyimpan informasi *layout* fisik dari chip yang di desain. Format ini menyimpan data mengenai posisi, bentuk, dan lapisan berbagai komponen IC, seperti transistor, jalur interkoneksi, dan elemen lainnya, yang diperlukan untuk proses fabrikasi. GDSII adalah format berbasis biner yang efisien dalam menangani *data layout* yang sangat kompleks dan besar, sehingga menjadi format utama yang digunakan oleh perusahaan fabrikasi semikonduktor (*foundries*) untuk memproduksi chip sesuai desain.

## D. *RTL-to-GDSII-Flow*

Alur desain chip dari RTL hingga menjadi layout fisik (GDSII) digambarkan pada Gambar 1. Proses dimulai dengan spesifikasi desain yang diimplementasikan dalam RTL menggunakan HDL (*Hardware Description Language*), seperti Verilog. Setelah verifikasi selesai hasilnya diteruskan ke tahap *logic design*. Di tahap ini, dapat digunakan *tool* Cadence Genus untuk melakukan sintesis logika guna menghasilkan *gate-level netlist*, yang kemudian dapat dicek *Logic Equivalence Check* (LEC) menggunakan Cadence Conformal. Apabila proses LEC selesai, proses berlanjut ke tahap *physical design* menggunakan Cadence Innovus untuk melakukan *placement* dan *routing* hingga menghasilkan layout fisik. Layout kemudian diverifikasi untuk memastikan fungsinya. Apabila semua proses verifikasi berhasil, hasil akhir akan berupa file GDSII yang dapat digunakan untuk fabrikasi chip. Jika ditemukan kesalahan di tiap tahap, proses dapat kembali ke langkah sebelumnya untuk melakukan perbaikan. Dalam *RTL-to-GDSII-Flow* ini terdapat beberapa *software* Cadence yang digunakan, antara lain:

### 1. Cadence Xcelium

Cadence Xcelium adalah *software* EDA (*Electronic Design Automation*) yang digunakan untuk simulasi desain sirkuit digital. Xcelium mendukung berbagai HDL seperti Verilog, VHDL, dan SystemVerilog, serta memungkinkan *behaviour test* pada level RTL (*Register Transfer Level*). *Software* ini digunakan untuk memverifikasi fungsionalitas, mendeteksi *bug*, dan memastikan bahwa desain memenuhi spesifikasi sebelum dilanjutkan ke tahap implementasi fisik. Dalam alur desain IC, Xcelium berperan penting dalam tahap verifikasi, memastikan desain sebelum tahap sintesis atau fabrikasi.

### 2. Cadence Genus

Cadence Genus adalah *software* EDA (*Electronic Design Automation*) yang digunakan dalam desain sirkuit digital untuk melakukan proses sintesis dari RTL (*Register Transfer Level*) ke *netlist* yang berbasis gerbang logika. *Software* ini mengubah kode RTL yang ditulis dalam *Hardware Description Language* (HDL) menjadi representasi fisik yang siap untuk langkah desain fisik selanjutnya. Genus memungkinkan desainer untuk

mengoptimalkan desain berdasarkan parameter seperti kecepatan, area, dan konsumsi daya, serta memastikan bahwa desain memenuhi batasan *timing*.

Dalam alur desain IC, Cadence Genus berperan sebagai tahap awal dari *RTL-to-GDSII flow*, di mana perangkat lunak ini menghasilkan *netlist* yang siap digunakan dalam tahapan *floorplanning* dan *placement-routing*.

## 3. Cadence Conformal

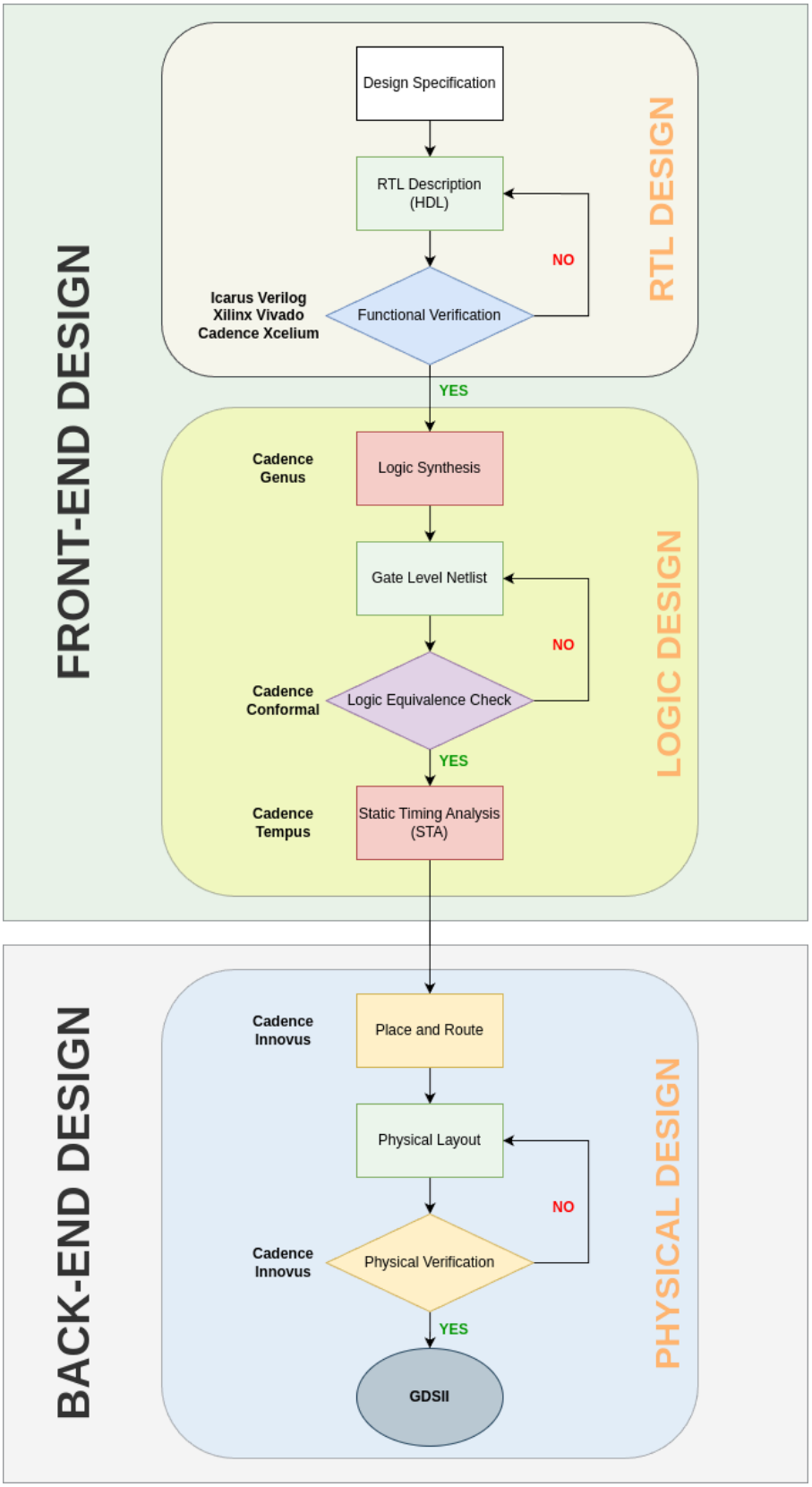
Cadence Conformal adalah software EDA (*Electronic Design Automation*) yang digunakan untuk memverifikasi kesetaraan fungsional antara desain logika pada level RTL (*Register Transfer Level*) dengan *netlist* hasil sintesis. *Tool* ini berperan penting dalam proses *formal verification* untuk memastikan tidak ada kesalahan logika yang terjadi selama proses sintesis atau optimisasi. Conformal mendukung berbagai metode pembuktian seperti equivalence checking dan lint checking, serta mampu menangani desain dengan kompleksitas tinggi. Dalam alur desain IC, Cadence Conformal membantu memastikan keandalan desain sebelum memasuki tahap implementasi lebih lanjut.

## 4. Cadence Tempus

Cadence Tempus adalah *software* EDA (*Electronic Design Automation*) yang digunakan untuk analisis *timing* dalam desain sirkuit digital. Tempus memfokuskan pada *Static Timing Analysis* (STA) untuk memastikan desain memenuhi batasan *timing* seperti *setup*, *hold*, *clock skew*, dan *clock uncertainty*. *Software* ini membantu desainer mengidentifikasi jalur \, menganalisis performa *clock*, dan mengoptimalkan desain untuk memenuhi target kecepatan tanpa melebihi batasan daya dan area.

## 5. Cadence Innovus

Cadence Innovus adalah software EDA (*Electronic Design Automation*) yang digunakan dalam proses *physical design layout* untuk menghasilkan *physical layout chip* dari *netlist* logika yang telah disintesis. Innovus memiliki peran dalam *floorplanning*, *placement*, *clock tree synthesis* (CTS), hingga *routing*, yang memastikan desain memenuhi batasan area, daya, dan *timing*. *Software* ini melakukan optimalisasi distribusi daya, mengelola jaringan *clock*, dan memastikan desain sesuai aturan manufaktur (DRC) serta performa di berbagai kondisi operasional (*multi-corner multi-mode*). Dalam alur desain IC, Cadence Innovus merupakan tahap penting pada RTL-to-GDSII flow, menghasilkan *layout* yang siap untuk fabrikasi.




**Gambar 1. Alur Diagram Proses Mengubah File RTL menjadi File GDSII**

# E. Hands-On

Dalam *hands-on* ini akan digunakan beberapa *library file* untuk memudahkan dalam proses RTL-to-GDSII yang akan dilakukan. Peserta diharap dapat men-*download file-file library* tersebut terlebih dahulu. *Library File* dapat di *download* disini.

# F. Logic Functional Verification

*Logic Verification* adalah proses untuk memastikan bahwa desain IC digital berfungsi sesuai spesifikasi logika yang diinginkan. Tujuan dari proses ini adalah mendeteksi dan memperbaiki *bug* pada tahap awal desain menggunakan simulasi. *Logic Verification* memastikan keandalan desain sebelum melanjutkan ke tahap implementasi fisik. Sebenarnya, simulasi menggunakan `iverilog` sudah cukup mewakili tahap ini. Namun, pada modul ini peserta akan dikenalkan dengan *software* Cadence yang dapat melakukan proses *logic verification*, yakni dengan menggunakan *software* Cadence Xcelium.

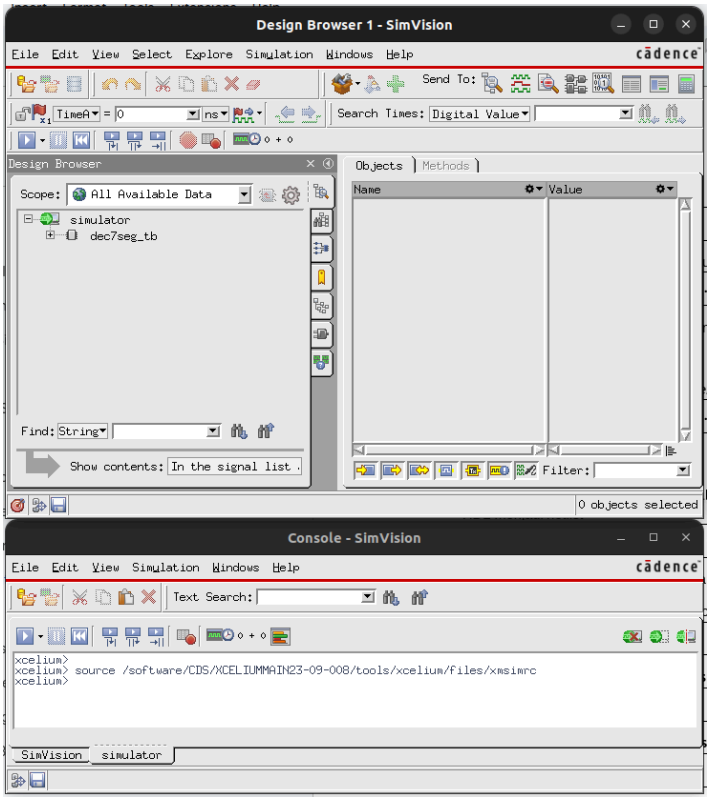
1. Masuk ke dalam folder **`sim/`**

```
cd sim
```

2. Gunakan perintah beriikut untuk melakukan *running simulation*

```
xrun ../rtl/[design].v ../tb/[test_bench].v -access +rwc -gui &
```

3. Akan terbuka jendela baru Cadence SimVision



4. Untuk melakukan *logic simulation*, dapat masuk ke menu `Wave from`




5. Untuk melakukan analisis *Coverage Verification* dapat digunakan perintah berikut

```
xrun ../rtl/[design].v ../tb/[test_bench].v -access +rwc -coverage
all -gui &
```

## G. Sintesis RTL

1. Pertama-tama buat sebuah direktori **`lib/`** yang akan berisi *library* untuk mensintesis HDL menjadi *netlist*

```
mkdir lib
```

2. Tambahkan *library* yang akan digunakan dalam *synthesis* ke dalam folder **`lib/`**

3. Dalam *root project*, Buat sebuah direktori baru bernama **`constraints/`**

```
mkdir constraints
```

4. Masuk ke dalam direktori **`constraints/`** tersebut

```
cd constraints
```

5. Buat *file* *.sdc* baru dan beri nama **`constraints.sdc`**

```
nano constraints.sdc
```

6. *Copy* isi *file* berikut ke dalam *file* **`constraints.sdc`** yang sudah dibuat

```
# input delay untuk iBCD
set_input_delay -max 2.0 [get_ports "iBCD"]

# output delay untuk oHEX
set_output_delay -max 1.0 [get_ports "oHEX"]

# output delay untuk anode
set_output_delay -max 1.0 [get_ports "anode"]
```

7. Kembali ke *root project*

```
cd ..
```

8. Buat sebuah direktori baru bernama **`synthesis/`**

```
mkdir synthesis
```

9. Buat *script* `.tcl` baru dan beri nama **`synthesis.tcl`**, untuk membantu dalam proses sintesis Verilog HDL

```
nano synthesis.tcl
```

10. Isi *script* **`synthesis.tcl`** yang sudah dibuat dengan *script* berikut

```
set_db init_lib_search_path ../lib/
set_db init_hdl_search_path ../rtl/
read_libs slow_vdd1v0_basiccells.lib

read_hdl dec7seg.v
elaborate
read_sdc ../constraints/constraints.sdc

set_db syn_generic_effort medium
set_db syn_map_effort medium
set_db syn_opt_effort medium

syn_generic
syn_map
syn_opt

# reports
report_timing -unconstrained > reports/report_timing.rpt
report_power  > reports/report_power.rpt
report_area   > reports/report_area.rpt
report_qor    > reports/report_qor.rpt

# outputs
write_hdl > outputs/dec7seg_netlist.v
write_sdc > outputs/dec7seg_sdc.sdc
write_sdf -timescale ns -nonegchecks -recrem split -edges check_edge  -setuphold
split > outputs/delays.sdf
```

11. *Run Script* TCL yang sudah dibuat

```
genus -f synthesis.tcl
```

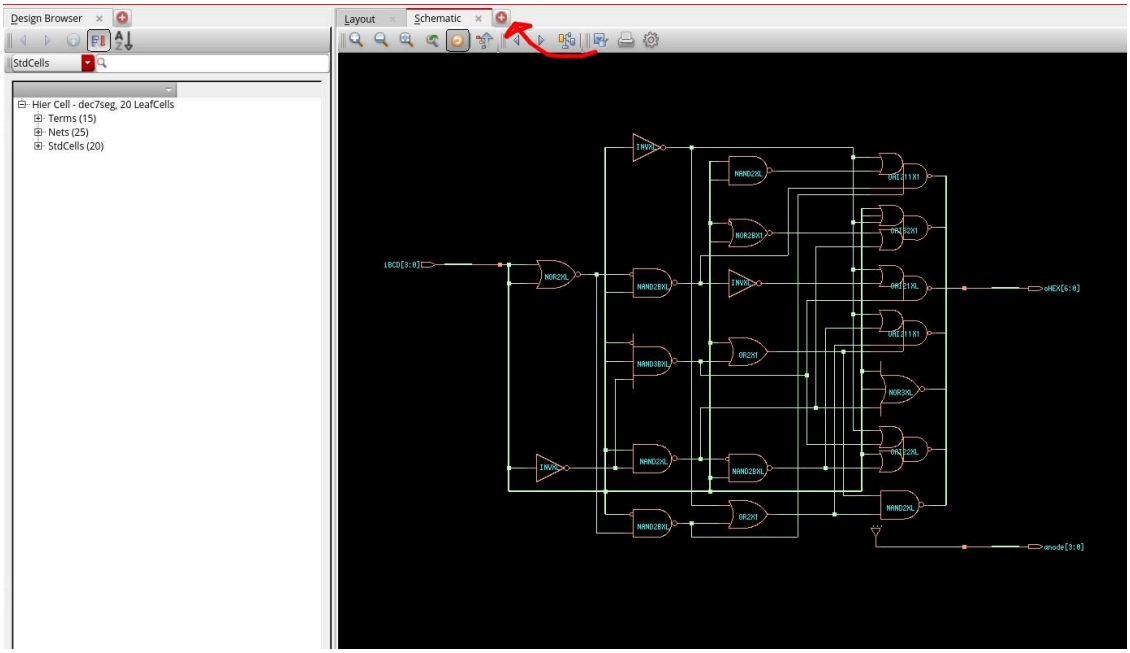
Jika program berhasil dijalankan, maka akan keluar informasi sebagai berikut:

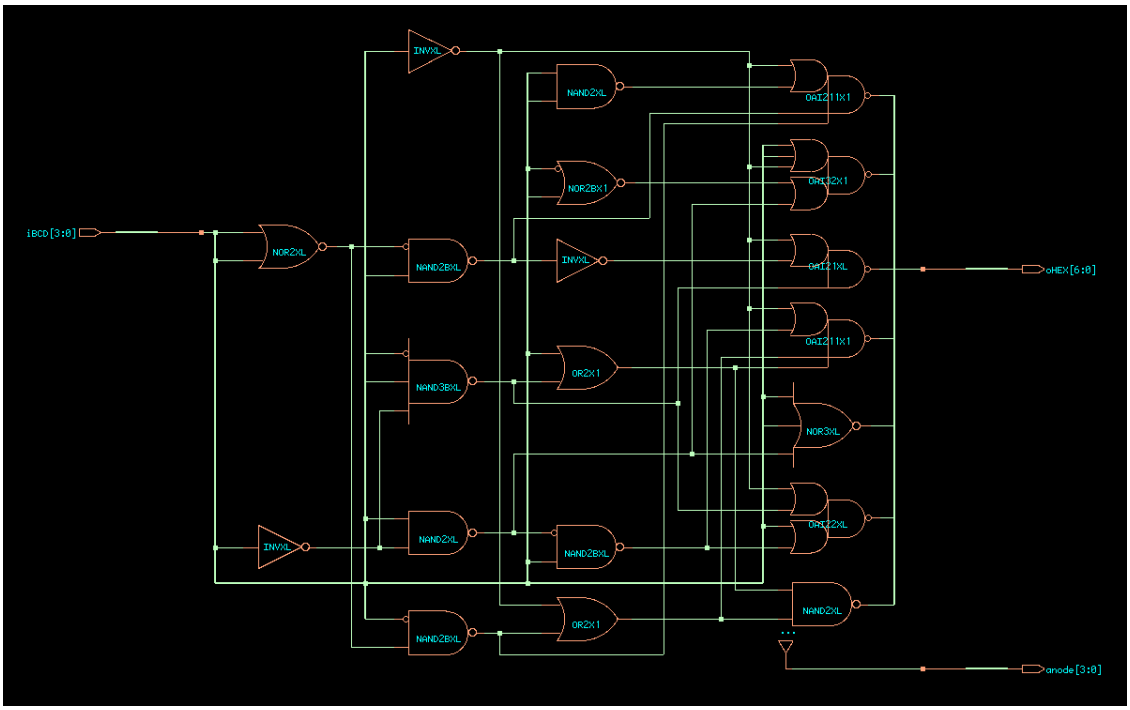
```
  =================
   Message Summary
  =================
----------------------------------------------------------------
|  Id   |Sev |Count|                 Message Text                 |
----------------------------------------------------------------
|CFM-1  |Info|    1|Wrote dofile.                                 |
|CFM-5  |Info|    1|Wrote formal verification information.        |
|PA-7   |Info|    4|Resetting power analysis results.             |
|       |    |     |All computed switching activities are removed.|
|SYNTH-5|Info|    1|Done mapping.                                 |
|SYNTH-7|Info|    1|Incrementally optimizing.                     |
----------------------------------------------------------------
Info    : Done incrementally optimizing. [SYNTH-8]
        : Done incrementally optimizing 'dec7seg'.
     flow.cputime  flow.realtime  timing.setup.tns  timing.setup.wns  snapshot
UM:*                                                                 syn_opt
```

12. Setelahnya, dapat dilihat hasil sintesis dengan menggunakan perintah berikut:

```
gui_show
```

13. Selanjutnya buka tab Schematic Viewer




14. Proses selesai, keluar dari Genus

```
exit
```

# H. *Report Analysis*

Setelah proses sintesis terdapat beberapa file .rpt (*report file*) yang berisi hasil yang dapat digunakan untuk analisis kualitas *logic synthesis*. Dalam implementasi berikut, terdapat 4 file .rpt yang dapat dianalisis, yakni `report_timing.rpt`, `report_power.rpt`, `report_area.rpt`, dan `report_qor.rpt`. Keempat file tersebut dihasilkan pada *script* TCL yang digunakan dalam synthesis menggunakan Genus. File-file tersebut dapat diakses pada direktori `synthesis/reports`.

```
# reports
report_timing > reports/report_timing.rpt
report_power  > reports/report_power.rpt
report_area   > reports/report_area.rpt
report_qor    > reports/report_qor.rpt
```

- **`report_timing.rpt`**
  Buka file dengan perintah berikut:

  ```
  cat report_timing.rpt
  ```

  Akan didapatkan hasil sebagai berikut:

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 22.15-s086_1
  Generated on:           Nov 18 2024  12:48:50 pm
  Module:                 dec7seg
  Operating conditions:   PVT_0P9V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================


Path 1: UNCONSTRAINED Late External Delay Assertion at pin oHEX[6]
     Startpoint: (F) iBCD[1]
       Endpoint: (R) oHEX[6]

                  Capture    Launch
      Drv Adjust:+       0         0
                                  -unconstrained
    Output Delay:-    1000
     Input Delay:-    2000
       Data Path:-     360

Exceptions/Constraints:
  input_delay              2000            constraints.sdc_line_2_2_1
  output_delay             1000            constraints.sdc_line_5

#-------------------------------------------------------------------------------------------
# Timing Point   Flags   Arc   Edge   Cell     Fanout Load Trans Delay Arrival Instance
#                                                (fF)  (ps)  (ps)   (ps)  Location
#-------------------------------------------------------------------------------------------
  iBCD[1]        -       -     F     (arrival)      4  0.8     0     0    2000    (-,-)
  g279__8246/Y   -       AN->Y F     NAND3BXL       3  0.6   170   162    2162    (-,-)
  g272__1617/Y   -       B->Y  F     OR2X1          2  0.5    30   165    2327    (-,-)
  g270__5107/Y   -       A->Y  R     NAND2XL        1  0.0    20    32    2360    (-,-)
  oHEX[6]        -       -     R     (port)         -    -     -     0    2360    (-,-)
#-------------------------------------------------------------------------------------------
```

*Report* ini menunjukkan analisis *timing path* antara titik awal (*startpoint*) dan titik akhir (*endpoint*) di desain `dec7seg`.

1. **Status *Timing Path***
   - *Path* ini merupakan *`UNCONSTRAINED`*, karena tidak ada *constraint clock* yang diterapkan.
   - *`Total Delay`* (*Input* + *Data Path* + *Output*): 2360 `ps`, berada dalam batas *delay eksternal* yang ditetapkan oleh SDC.
2. ***Path* yang Dianalisis**
   - *`Startpoint`*: Sinyal *input* `iBCD[1]` dengan transisi *falling edge* (`F`).
   - *`Endpoint`*: Sinyal *output* `oHEX[6]` dengan transisi *rising edge* (`R`).
   - *path* ini adalah *path* kombinasi, karena desain tidak memiliki *elemen clock* ataupun register.
3. **Delay Eksternal dan Internal**
   - *`Input Delay`*: 2000 `ps`, menunjukkan waktu propagasi maksimum sinyal dari port `iBCD` hingga diterima oleh desain internal.
   - *`Output Delay`*: 1000 `ps`, menunjukkan waktu maksimum yang diizinkan untuk sinyal dari desain internal hingga mencapai *port output* `oHEX`.
   - *`Data Path Delay`*: 360 `ps`, waktu total yang diperlukan untuk propagasi sinyal dari `iBCD[1]` ke `oHEX[6]` melalui jaringan logika internal.
4. **Detail *Timing Point***
   - *`Fanout`*: Menunjukkan jumlah elemen logika yang digerakkan oleh sinyal. Seperti, `iBCD[1]` memiliki *fanout* sebesar 4.
   - *`Load`*: Kapasitansi yang digerakkan oleh setiap elemen logika, dalam *femtofarad* (`fF`).
   - *`Transition Time`*: Waktu transisi sinyal di setiap titik (dalam `ps`).
   - *`Delay`*: Waktu propagasi sinyal antara titik-titik jaringan.
     *Path* terdiri dari elemen berikut:
     a. *`iBCD[1]`* → *Port input* dengan *arrival time* awal 2000 `ps`.
     b. *`g279__8246/Y`* → Gerbang `NAND3BXL` dengan *fanout* 3 dan delay 170 `ps`.
     c. *`g272__1617/Y`* → Gerbang `OR2X1` dengan *fanout* 2 dan delay 30 `ps`.
     d. *`g270__5107/Y`* → Gerbang `NAND2XL` dengan *fanout* 1 dan delay 20 `ps`.
     e. *`oHEX[6]`* → *Port output* dengan total waktu *arrival time* 2360 `ps`.

- **`report_power.rpt`**

  Buka file dengan perintah berikut:

```
cat report_power.rpt
```

  Akan didapatkan hasil sebagai berikut:

```
Instance: /dec7seg
Power Unit: W
PDB Frames: /stim#0/frame#0
  ------------------------------------------------------------------------
    Category         Leakage     Internal    Switching        Total    Row%
  ------------------------------------------------------------------------
       memory     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
     register     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
```

```
      latch     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
      logic     4.92852e-10  1.89073e-07  9.51750e-08  2.84741e-07 100.00%
       bbox     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
      clock     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
        pad     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
         pm     0.00000e+00  0.00000e+00  0.00000e+00  0.00000e+00   0.00%
  ------------------------------------------------------------------------
    Subtotal     4.92852e-10  1.89073e-07  9.51750e-08  2.84741e-07 100.00%
  Percentage           0.17%       66.40%       33.43%      100.00% 100.00%
  ------------------------------------------------------------------------
```

Berdasarkan hasil *report* menunjukkan penggunaan daya berdasarkan kategori komponen dalam desain modul `dec7seg`. Dari tabel, dapat dilihat bahwa semua daya konsumsi berasal dari komponen *`logic`*, dengan total daya sebesar 2.84741e-07 W (284.741 nW). Konsumsi daya ini terbagi menjadi tiga komponen utama:

1. ***Leakage Power* (0.17%)**
   Sebesar **`4.92852e-10 W (0.49285 nW)`**, merupakan daya yang hilang akibat kebocoran transistor dalam kondisi statis.
2. ***Internal Power* (66.40%)**
   Sebesar **`1.89073e-07 W (189.073 nW)`**, berasal dari aktivitas internal sel logika seperti pengisian dan pengosongan kapasitansi internal.
3. ***Switching Power* (33.43%)**
   Sebesar **`9.51750e-08 W (95.175 nW)`**, berasal dari pengalihan kapasitansi pada node output selama transisi logika.

Kategori lain, seperti memori, register, latch, clock, dan pad, tidak berkontribusi terhadap konsumsi daya karena modul `dec7seg` ini adalah desain kombinasi tanpa elemen *domain clock*. Dari hasil ini, dapat disimpulkan bahwa daya utama desain berasal dari aktivitas *switching* dan internal logika.

- **`report_area.rpt`**
  Buka file dengan perintah berikut:

```
cat report_area.rpt
```

Akan didapatkan hasil sebagai berikut:

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 22.15-s086_1
  Generated on:           Nov 18 2024  12:48:50 pm
  Module:                 dec7seg
  Operating conditions:   PVT_0P9V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Instance Module  Cell Count  Cell Area  Net Area   Total Area   Wireload
-----------------------------------------------------------------------------
dec7seg                  20     27.360     0.000       27.360 <none> (D)
  (D) = wireload is default in technology library
```

Hasil analisis menunjukkan bahwa modul `dec7seg` menggunakan sebanyak `20 sel`, dengan total luas area sel sebesar `27.360` (dalam satuan teknologi yang digunakan, seperti µm²). Tidak ada area tambahan yang dialokasikan untuk interkoneksi jaringan (*net area*), sehingga total area desain hanya mencakup penggunaan sel logika. Model *wireload* yang

diterapkan adalah model default dari library teknologi, yang tidak mempertimbangkan jalur spesifik atau layout fisik desain. Hasil ini mencerminkan bahwa `dec7seg` adalah modul yang ringan dan efisien, dengan area yang sederhana karena desainnya murni kombinasi tanpa *elemen clock*.

- **`report_qor.rpt`**
  Buka file dengan perintah berikut:

```
cat report_qor.rpt
```

  Akan didapatkan hasil sebagai berikut:

```
============================================================
  Generated by:           Genus(TM) Synthesis Solution 22.15-s086_1
  Generated on:           Nov 18 2024  12:48:50 pm
  Module:                 dec7seg
  Operating conditions:   PVT_0P9V_125C (balanced_tree)
  Wireload mode:          enclosed
  Area mode:              timing library
============================================================

Timing
-------
  Cost    Critical         Violating
 Group   Path Slack  TNS     Paths
-------------------------------------
default    No paths   0.0
-------------------------------------
Total                 0.0          0

Instance Count
--------------
Leaf Instance Count             20
Physical Instance count          0
Sequential Instance Count        0
Combinational Instance Count    20
Hierarchical Instance Count      0

Area
----
Cell Area                          27.360
Physical Cell Area                 0.000
Total Cell Area (Cell+Physical)    27.360
Net Area                           0.000
Total Area (Cell+Physical+Net)     27.360

Max Fanout                         7 (iBCD[2])
Min Fanout                         1 (n_1)
Average Fanout                     2.4
Terms to net ratio                 2.7600
Terms to instance ratio            3.4500
Runtime                            6.981465 seconds
Elapsed Runtime                    81 seconds
Genus peak memory usage            1774.00
Innovus peak memory usage          no_value
Hostname                           ICDEC4
```

*Report* ini merangkum berbagai metrik analisis desain untuk modul `dec7seg`, meliputi timing, jumlah instance, area, fanout, serta performa selama proses sintesis. Secara rinci sebagai berikut:

1. ***Instance Count***
   - *`Leaf Instance Count`*: 20, mencerminkan jumlah elemen logika dasar yang digunakan.
   - *`Combinational Instance Count`*: 20, menunjukkan bahwa desain ini sepenuhnya terdiri dari logika kombinasi tanpa elemen sekuensial seperti flip-flop atau register.
   - *`Hierarchical Instance Count`*: 0, menunjukkan desain ini tidak memiliki instans hierarkis atau modul terpisah lainnya.
2. ***Area***
   - *`Cell Area`*: 27.360, total area logika yang digunakan oleh sel dasar.
   - *`Net Area`*: 0.000, menunjukkan area interkoneksi tidak dihitung atau tidak signifikan.
   - *`Total Area`*: 27.360, yang hanya mencakup area sel logika karena tidak ada elemen fisik tambahan.
3. ***Fanout Max***
   - *`Fanout`*: 7, terjadi pada sinyal `iBCD[2]`, menunjukkan bahwa sinyal ini menggerakkan 7 elemen logika.
   - *`Min Fanout`*: 1, terjadi pada sinyal `n_1`, yang hanya menggerakkan satu elemen logika.
   - *`Average Fanout`*: 2.4, mencerminkan rata-rata beban setiap sinyal dalam desain.
4. **Performa Sintesis**
   - Waktu eksekusi sintesis adalah 6.981465 detik, dengan waktu total 81 detik, menunjukkan proses yang efisien untuk desain sederhana ini.
   - *Genus peak memory usage* adalah 1774 MB.

# I. *Logic Equivalence Check* (LEC)

*Logic Equivalence Check* (LEC) adalah proses verifikasi dalam desain IC digital untuk memastikan bahwa dua representasi desain, RTL (*Register Transfer Level*) dan *netlist* hasil sintesis, memiliki perilaku logika yang sama. LEC biasanya digunakan setelah sintesis atau optimasi desain untuk memastikan bahwa perubahan dalam proses tersebut tidak mengubah fungsionalitas desain. Proses ini dilakukan dengan membandingkan dua desain menggunakan algoritma formal tanpa menjalankan simulasi. LEC sangat penting dalam alur desain IC untuk menjamin konsistensi antara langkah-langkah desain, sehingga mengurangi risiko kesalahan pada tahap fabrikasi. Berikut adalah langkah-langkah proses LEC menggunakan *software* Cadence Conformal:

1. Dalam *root project*, buat direktori baru bernama ***`lec/`***

```
mkdir lec
```

2. Masuk ke dalam direktori tersebut

```
cd lec
```

3. Buat file *.do* kemudian *copy script* berikut

```
set log file dec7seg.log -replace
read library ../lib/slow_vdd1v0_basiccells.v -verilog -both
read design ../rtl/dec7seg.v -verilog -golden
read design ../synthesis/outputs/dec7seg_netlist.v -verilog -revised
add pin constraints 0 SE  -revised
add ignored inputs scan_in -revised
add ignored outputs scan_out -revised
set system mode lec
add compared point -all
compare
```

4. *Run script* berikut di terminal

```
lec -XL -nogui -color -64 -dofile dec7seg.do
```

5. Akan didapatkan hasil sebagai berikut

```
================================================================================
Mapped points: SYSTEM class
--------------------------------------------------------------------------------
Mapped points     PI     PO        Total
--------------------------------------------------------------------------------
Golden            4      11        15
--------------------------------------------------------------------------------
Revised           4      11        15
================================================================================
// Command: add compared point -all
// 11 compared points added to compare list
// Command: compare
================================================================================
Compared points      PO        Total
--------------------------------------------------------------------------------
Equivalent           11        11
================================================================================
```

15. Proses selesai, keluar dari Conformal

```
exit
```

# J. *Static Timing Analysis*

*Static Timing Analysis* (STA) adalah metode verifikasi desain sirkuit digital untuk memastikan bahwa sinyal dalam desain memenuhi batasan *timing*, seperti *setup time* dan *hold time*, tanpa perlu menjalankan simulasi. STA menganalisis semua *logic path* dalam desain secara matematis untuk mendeteksi potensi kesalahan *timing*, seperti penundaan sinyal yang dapat menyebabkan kesalahan operasi. Proses ini mencakup analisis pada *critical path*, *clock skew*, dan *clock uncertainty*. STA merupakan langkah penting dalam alur desain IC untuk memastikan bahwa desain akan berfungsi pada kecepatan yang diinginkan sebelum fabrikasi.

1. Dalam *root project*, buat direktori baru bernama **`sta/`**

```
mkdir sta
```

2. Masuk ke dalam direktori tersebut

```
cd sta
```

3. Buat sebuah file baru bernama `script.tcl`

```
nano script.tcl
```

4. Kemudian, *copy* kode berikut ke dalam *file* `script.tcl` tersebut

```
file mkdir reports

read_lib ../lib/slow_vdd1v0_basiccells.lib
read_verilog ../synthesis/outputs/dec7seg_netlist.v

set_top_module dec7seg

read_sdc ../synthesis/outputs/dec7seg_sdc.sdc

# report
check_timing > reports/check_timing.rpt
report_timing > reports/timing_report.rpt
report_analysis_coverage > reports/analysis_coverage.rpt
report_analysis_summary > reports/analysis_summary.rpt
report_annotated_parasitics > reports/annotated.rpt

report_clocks > reports/clocks.rpt
report_case_analysis > reports/case_analysis.rpt
report_constraints -all_violators > reports/allviolations.rpt

#w ireload report
report_wire_load > reports/wireload.rpt

report_timing -retime path_slew_propagation -max_path 30 -nworst 30 -path_type
full_clock > reports/pba.rpt
```

```
report_timing -late -max_paths 30 -nworst 30 -path_type full_clock -net >
reports/gba_setup.rpt
report_timing -early -max_paths 30 -nworst 30 -path_type full_clock -net >
reports/gba_hold.rpt

report_timing -max_paths 30 -nworst 30 -retime path_slew_propagation >
reports/pba_setup.rpt
report_timing -early -max_path 30 -nworst 30 -retime path_slew_propagation >
reports/pba_hold.rpt
```

5. Kemudian jalankan *script* tersebut dengan perintah berikut

```
tempus -nowin -init script.tcl
```

6. Maka proses STA akan berjalan, dan hasil *report* nya dapat dilihat dan dianalisis pada direktori **`reports/`**. Proses selesai *Static Timing Analysis* selesai, keluar dari Tempus

```
exit
```

# K. *Physical Design*

1. Pertama-tama buat sebuah direktori **`lef/`** yang akan berisi *library* untuk membantu dalam proses *Place & Route*

```
mkdir lef
```

2. Tambahkan *library* LEF *file* ke dalam folder **`lef/`**
3. Buat direktori **`pd/`** yang berarti *Physical Design* untuk memudahkan dalam menyimpan *log-log* saat proses *physical design* terjadi

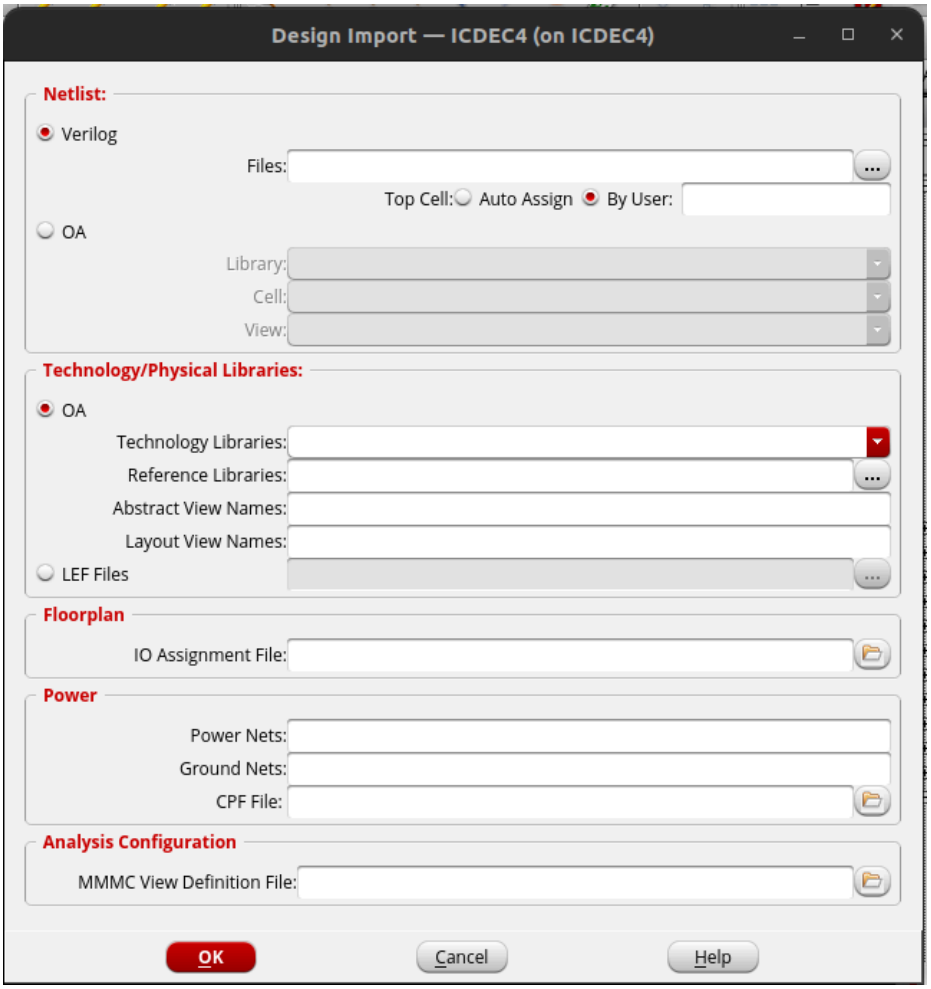
```
mkdir pd
```

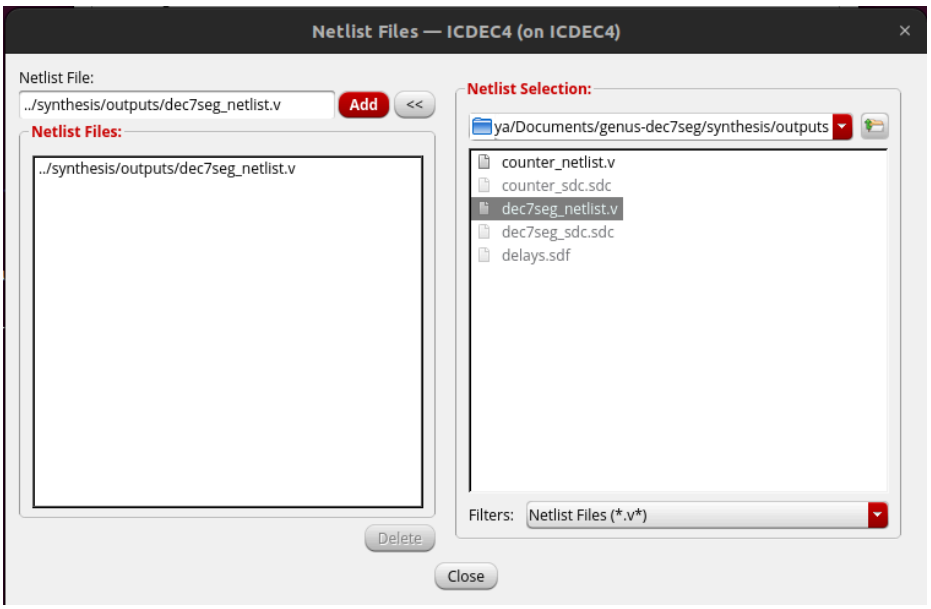
4. Masuk kedalam direktori tersebut kemudian buka *Software* Cadence Innovus

```
cd pd && innovus -stylus
```

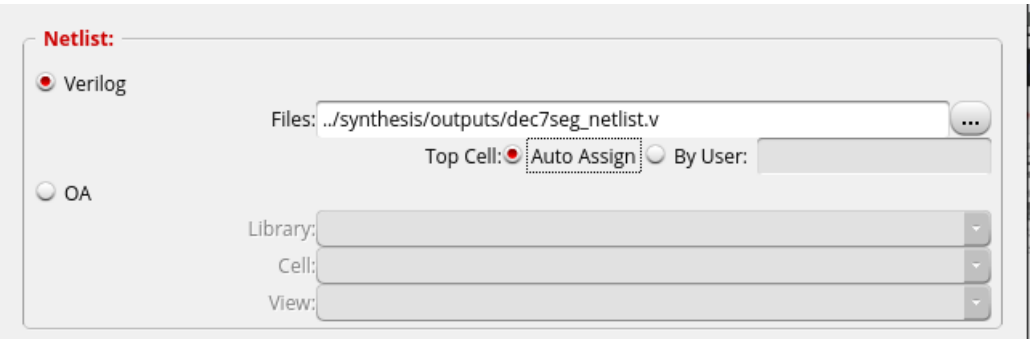
5. Apabila Innovus sudah terbuka. Klik `File -> Import Design` untuk menggunakan *file netlist* yang sudah dirancang sebelumnya kedalam *layout*

6. Pada bagian Netlist, pilih Verilog. Kemudian pilih files dan pilih netlist yang sebelumnya di *generate*. Klik *Add* kemudian *Close*.

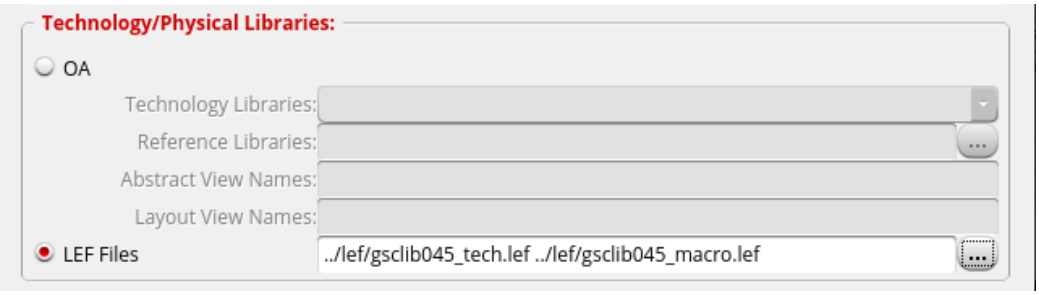


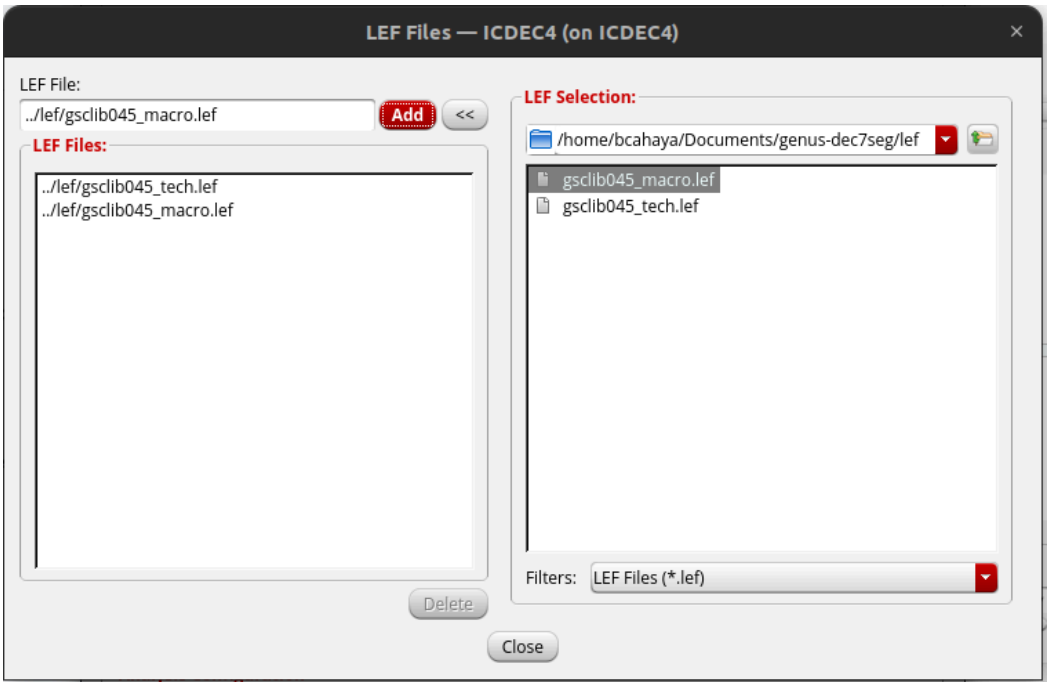



7. Pada bagian `Top Cell`, pilih `Auto Assign`




8. Pilih `LEF Files` pada bagian `Technology/Physical Libraries` kemudian pilih kedua *LEF files* yang sebelumnya ditambahkan dalam direktori **`lef/`**



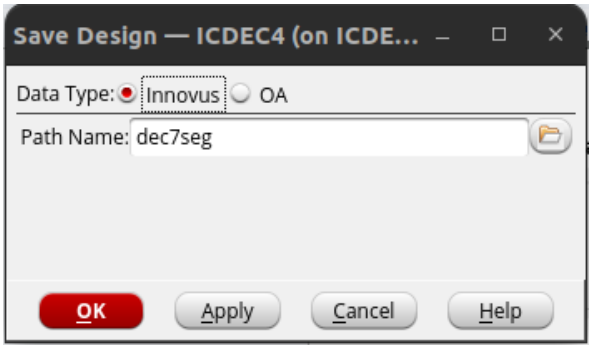



9. Pada bagian Power isikan VDD pada Power Nets dan GND pada Ground Nest.




10. Jika sudah *klik* OK.

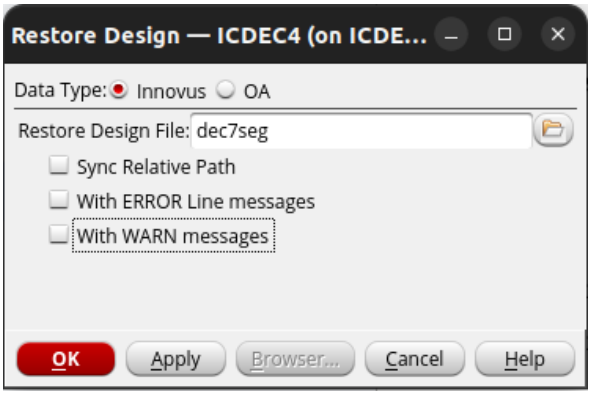
11. Simpan *Design* dengan cara pilih File -> Save Design. Kemudian pilih Innovus pada bagian Data Type. Dan beri nama yang sesuai. Jika sudah *klik* OK.



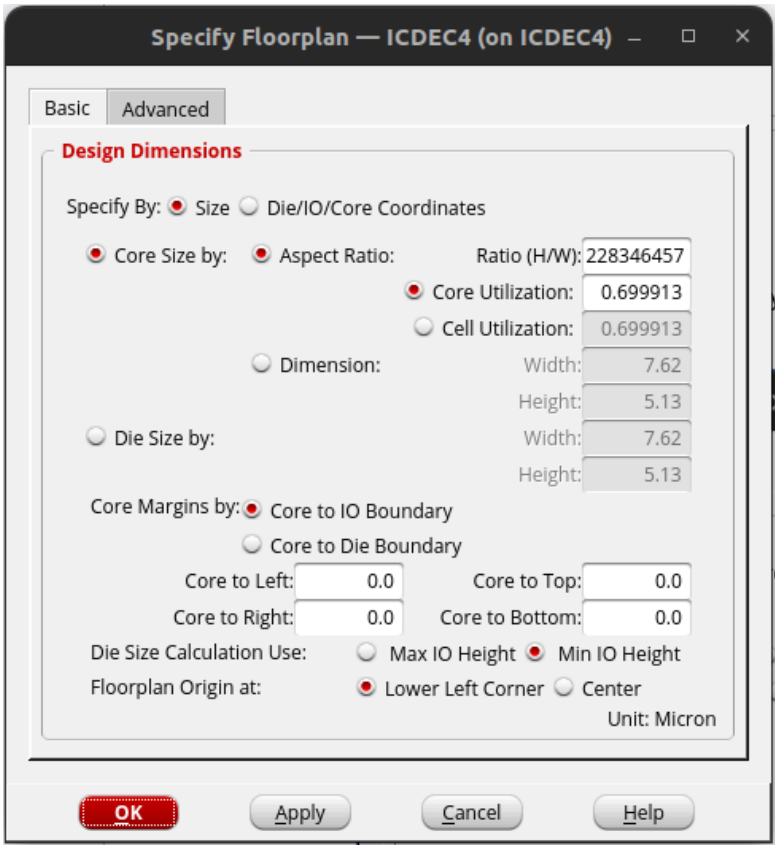

** note :*

- *Jangan lupa simpan design yang dirancang secara berkala*

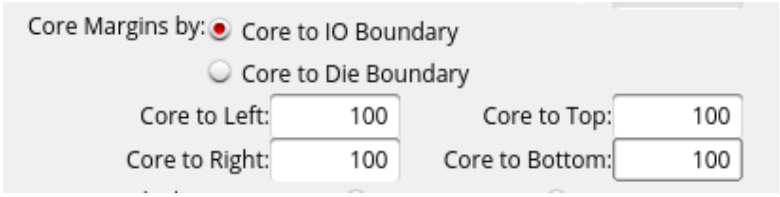
12. Untuk me-*load design* yang sudah disimpan. pilih File -> Restore Design. Pilih Innovus pada bagian Data Type. Selanjutnya pilih *design folder* yang sudah di simpan. Jika sudah *klik* OK.



13. Selanjutnya, pilih `Floorplan -> Specify Floorplan` untuk membuka menu *`Specify Floorplan`*.




14. Pada bagian *Tab* `Basic` pilih `Core to IO Boundary` pada bagian `Core Margins`. Kemudian *set* semua *margin* nya ke nilai 100.

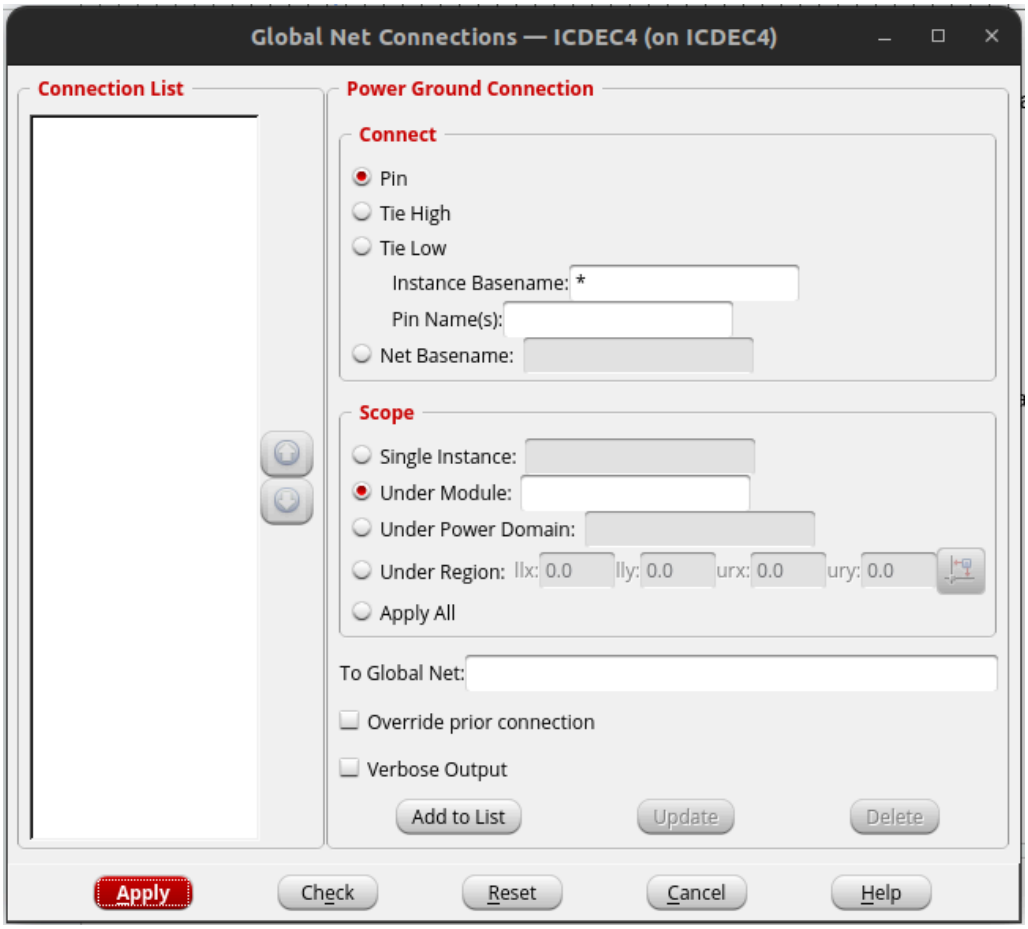



** note :*

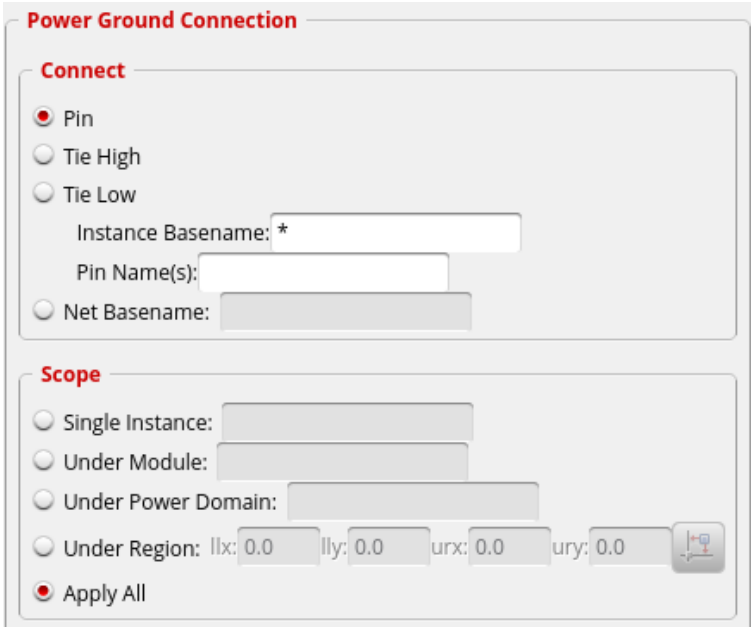
- *Ukuran margin dapat disesuaikan dengan design yang peserta rancang*
- *Dalam modul digunakan margin dengan nilai 100, namun tampaknya terlalu berlebihan sehingga hasilnya terlalu besar*
- *Peserta dapat menyesuaikan kembali dengan design yang telah peserta rancang*

15. Jika sudah *klik* `OK`.

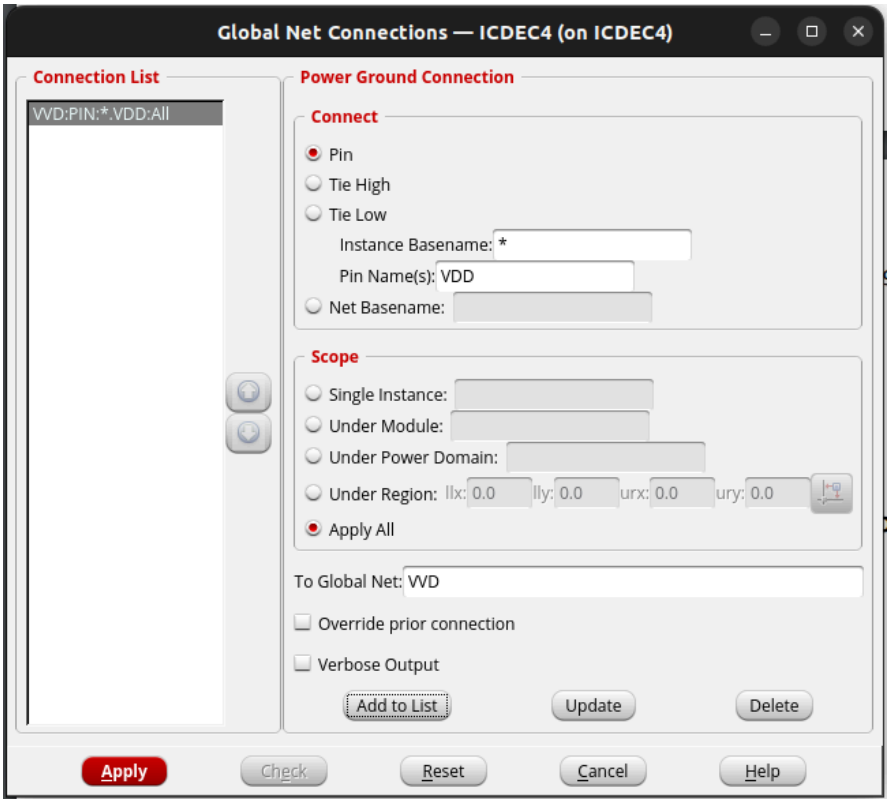
16. Kembali pada Innovus Tool Menu Bar. Pilih Power -> Connect Global Nets.




17. Di `Power Ground Connection`, pilih `Pin` pada bagian `Connect` dan pilih `Apply All` pada `Scope`.



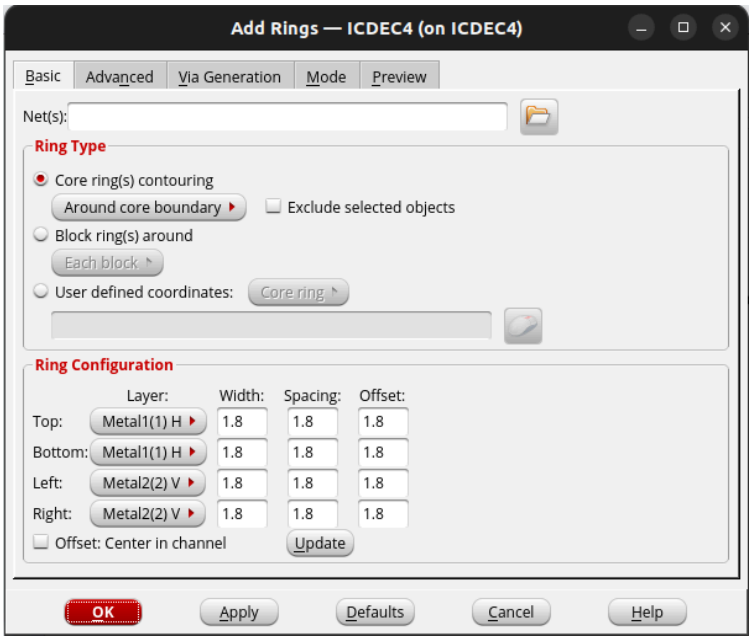
18. Untuk setiap *net* `VDD` dan `GND` yang tadi sudah dideklarasikan. Masukkan *net name* nya kedalam *box* `To Global Net` dan `Pin Names(s)`. Kemudian pilih `Add to List`.



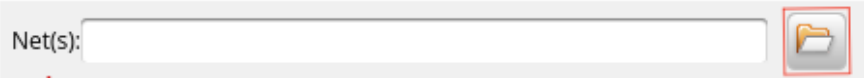



19. Jika sudah klik `Apply`, kemudian tutup *dialog box* nya.

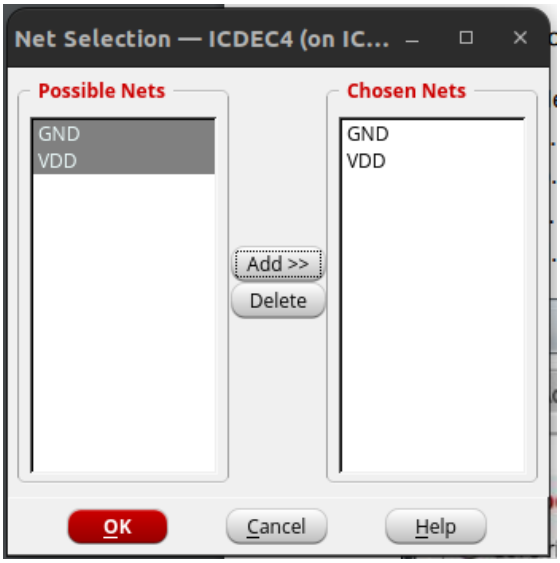
20. Selanjutnya pilih `Power -> Power Planning -> Add Ring` pada Innovus *tool menu bar*.




*21. Net* `VDD` dan `GND` yang sudah dideklarasikan tadi akan dimasukkan ke dalam `Ring`. Terlebih dahulu *klik icon folder* di sebelah kolom `Net` untuk memasukkan *net* yang dibutuhkan.




22. Pilih *net* `VDD` dan `GND` dari bagian `Possible Nets Column` kemudian pilih `Add` agar bisa ter *copy* ke dalam `Chosen Nets`. Jika sudah klik `OK`.



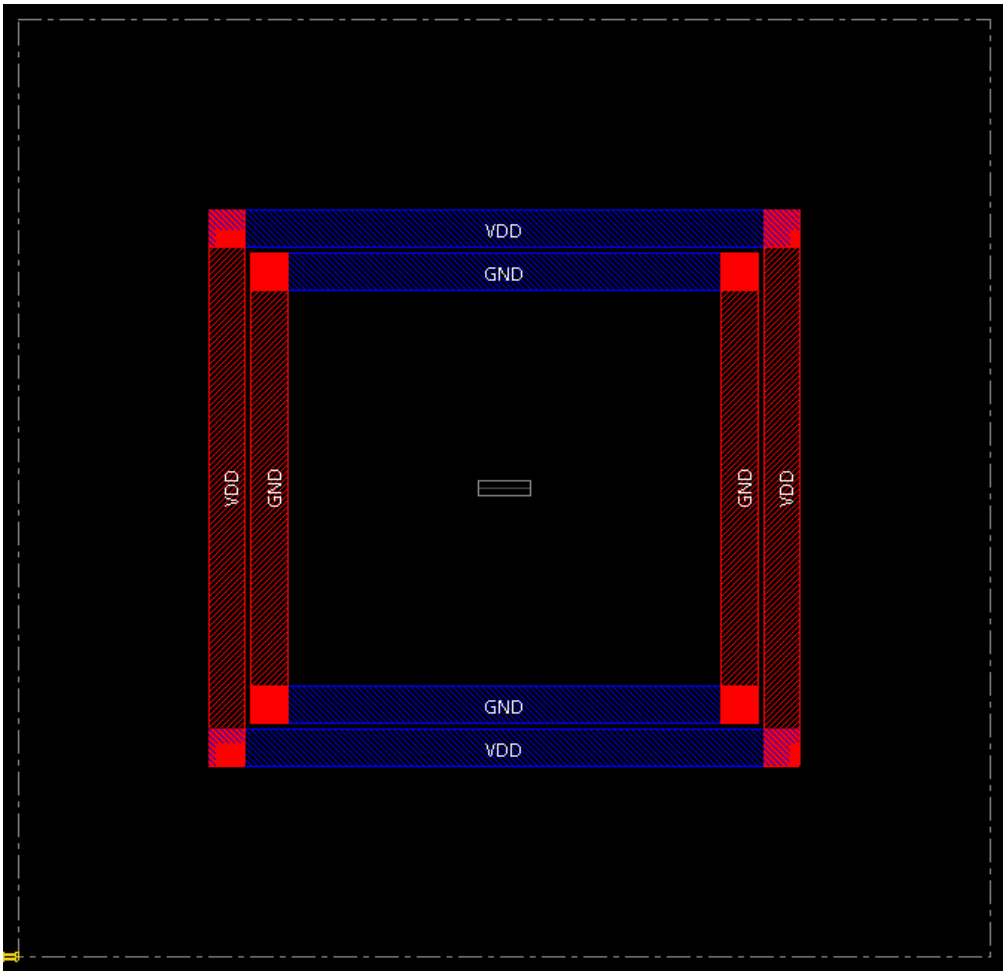
23. Pada bagian `Ring Configuration`, pilih `metal1` untuk *`Top`* dan *`Bottom`* dan `metal2` untuk `Left` dan `Right`. Isi bagian `Width` menjadi 8 dan `Spacing` menjadi 1,25. Kemudian, centang *box* `Offset: Center in channel`.



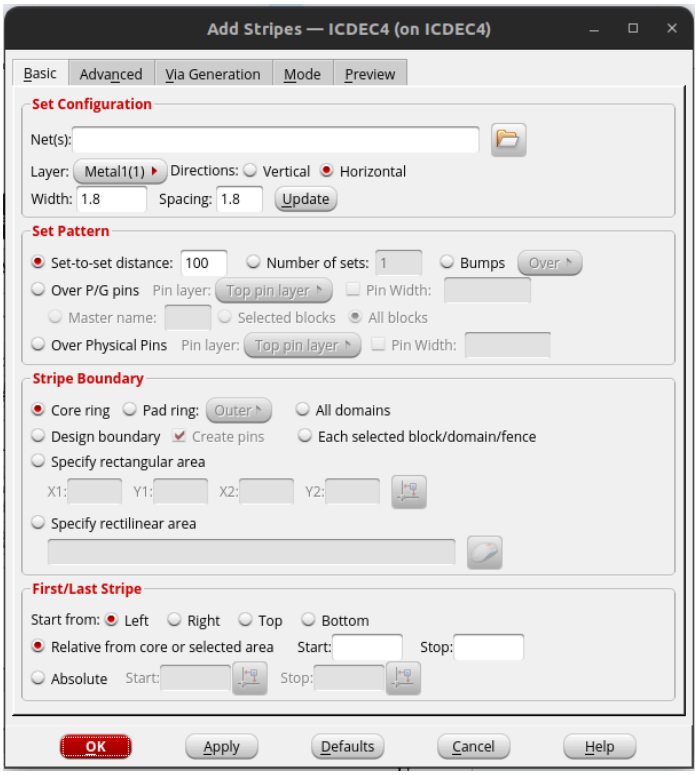

** note :*

- *Ukuran width dan spacing dapat disesuaikan dengan design yang peserta rancang*
- *Dalam modul digunakan width dengan nilai 8 dan spacing dengan nilai 1.25, namun tampaknya terlalu berlebihan sehingga hasilnya terlalu besar*
- *Peserta dapat menyesuaikan kembali dengan design yang telah peserta rancang*

24. Kemudian klik `OK`.



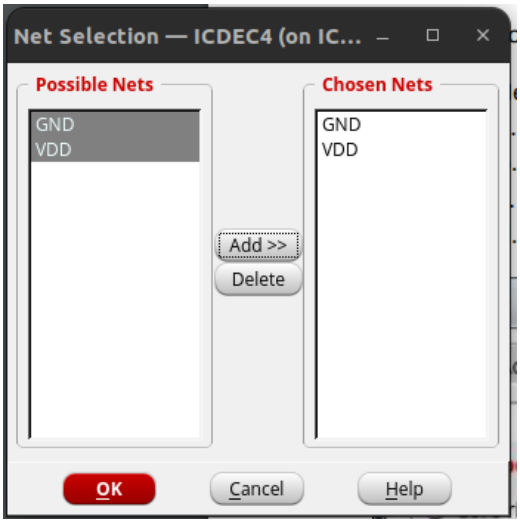
25. Selanjutnya pilih `Power -> Power Planning -> Add Stripes` pada Innovus *tool menu bar*.



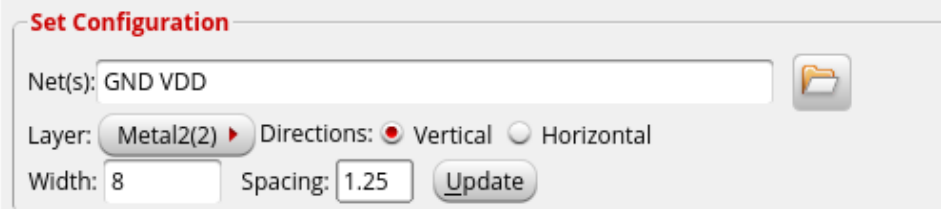

26. Pada *Tab Basic*. *Net* `VDD` dan `GND` yang sudah dideklarasikan tadi akan dimasukkan ke dalam `Ring`. Terlebih dahulu *klik icon folder* di sebelah kolom `Net` untuk memasukkan *net* yang dibutuhkan.




27. Pilih *net* `VDD` dan `GND` dari bagian `Possible Nets Column` kemudian pilih `Add` agar bisa ter *copy* ke dalam `Chosen Nets`. Jika sudah klik `OK`.



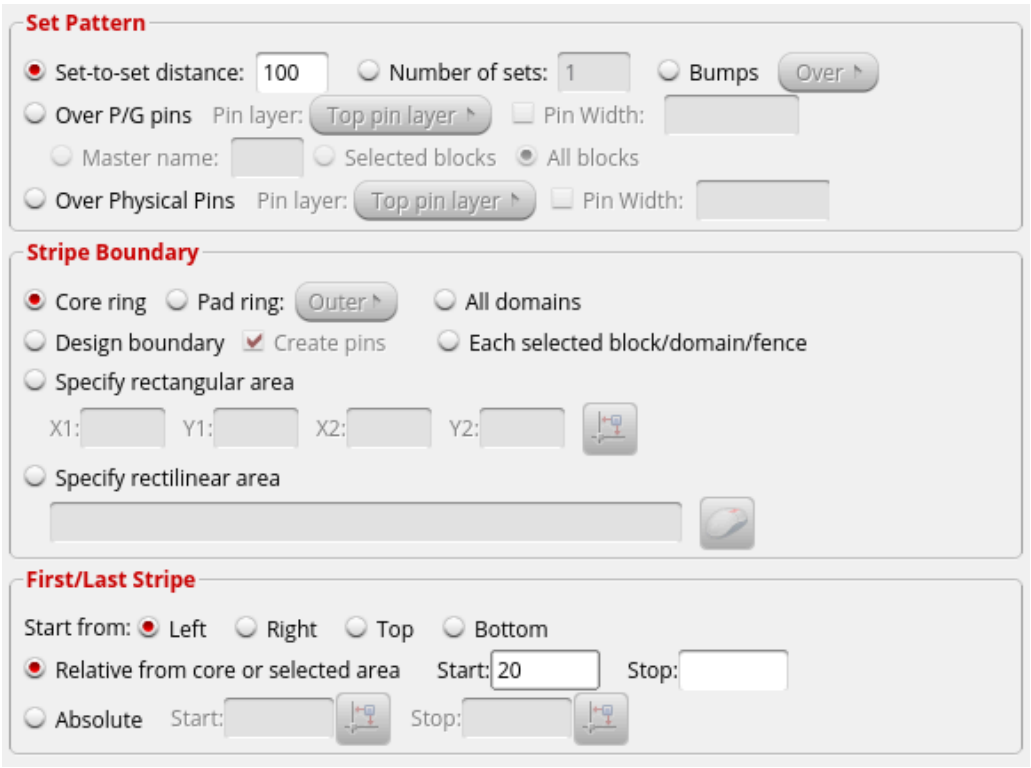
28. Di bagian `Set Configuration`, pilih `Layer Metal2` dan `Direction Vertical`. Kemudian ubah nilai `Width` menjadi 8 dan `Spacing` menjadi 1.25.



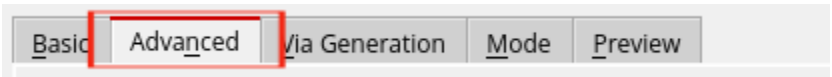

** note :*

- *Sesuaikan nilai width dan spacing pada step no 28 ini dengan nilai width dan spacing pada step 23*

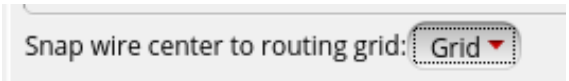
29. Ke bagian `Set Pattern`, ubah nilai `Set-to-set distance` menjadi 100. Kemudian pada `First/Last Stripe`, pilih `Relative from core or selected area`, beri nilai `Start` nya menjadi 20.



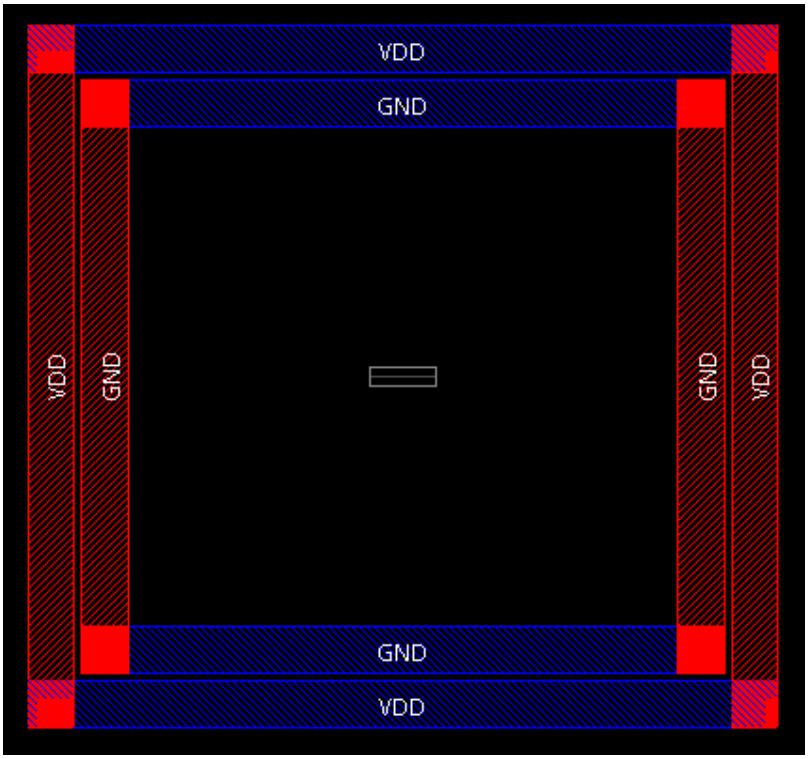

30. Pindah kebagian Tab `Advance`



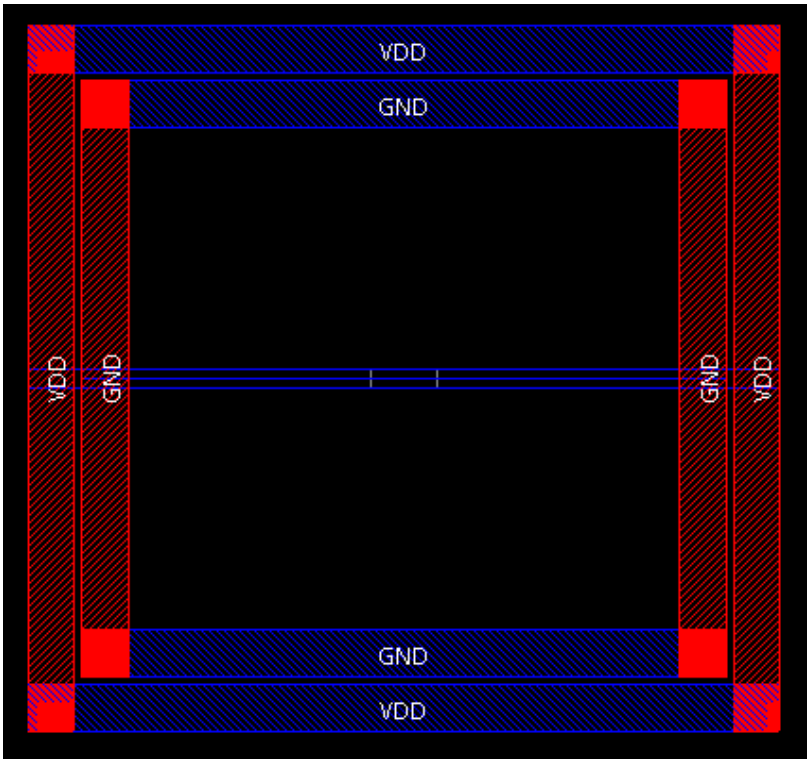

31. Pada bagian `Snap wire center to routing grid` pilih opsi `Grid`



32. Kemudian klik `OK`.



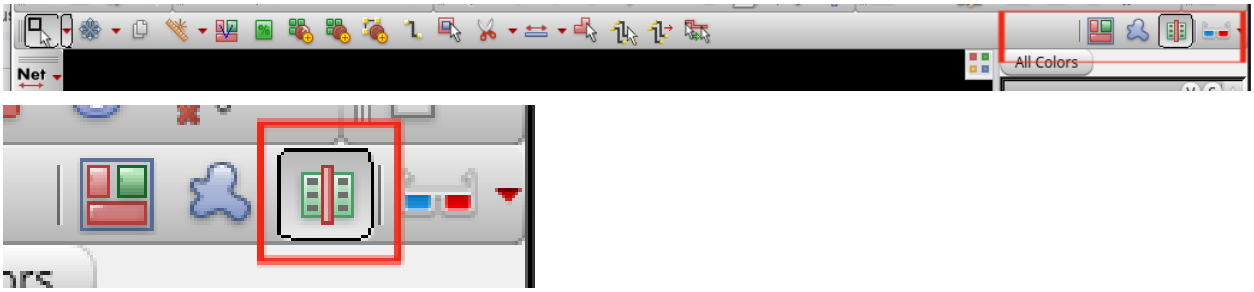

33. Pada Innovus *tool menu bar*, pilih `Route -> Special Route`. Kemudian klik `OK`. Proses ini akan membuat `VDD` dan `GND` untuk baris *standard cell* yang dirancang.



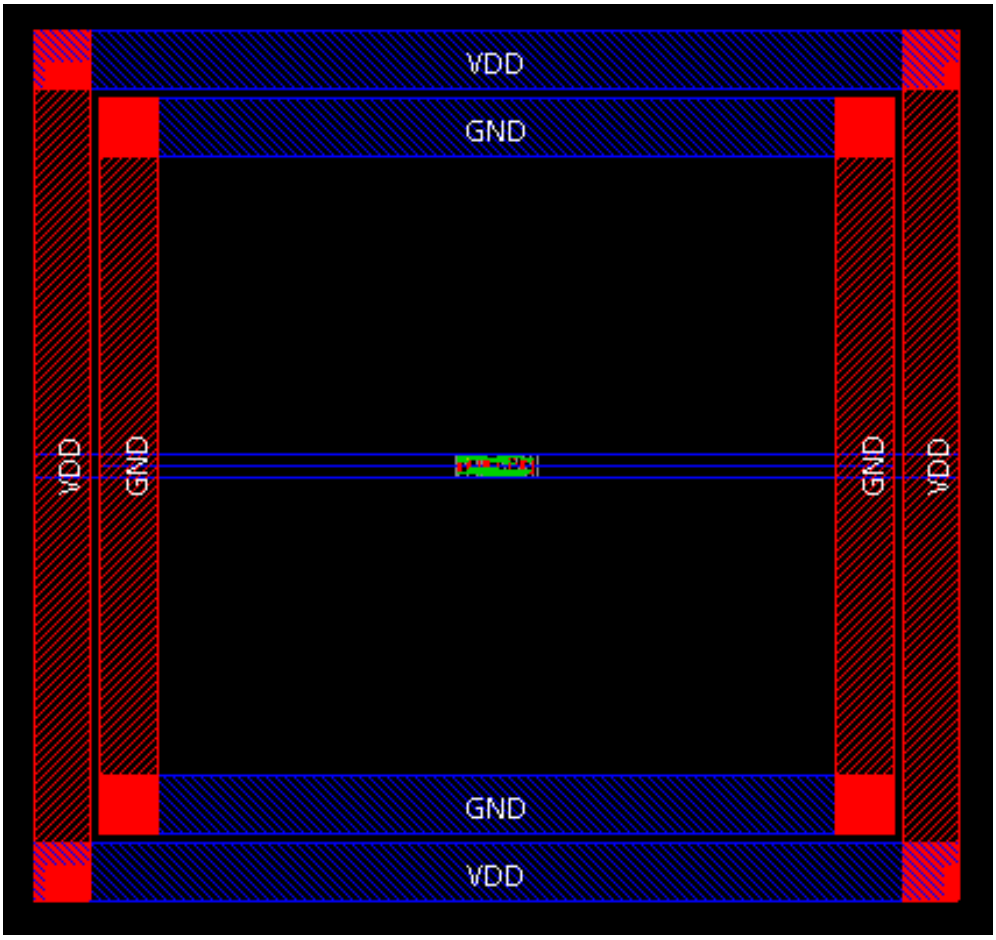
34. Pada Innovus *tool menu bar*, pilih `Place -> Place Standard Cell`. Kemudian pilih `Run Full Placement`. Centang *box* `Include Pre-Place Optimization`. Selanjutnya klik `OK`.

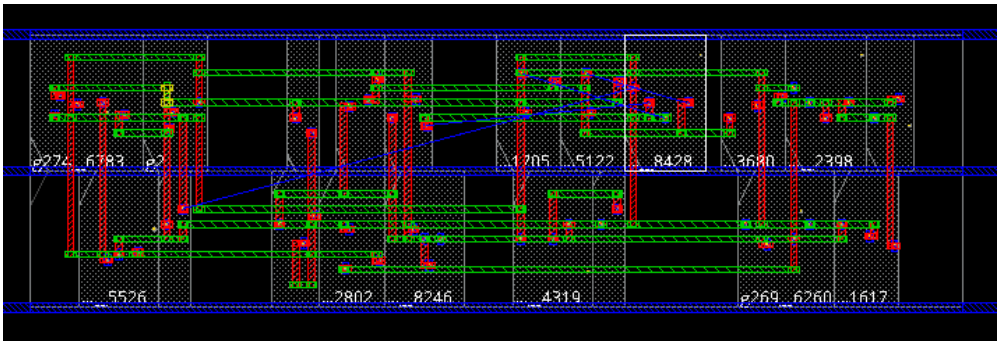



35. Klik Physical View pada tombol berikut:



36. Maka tampilan *Physical Design* dapat tertampil.

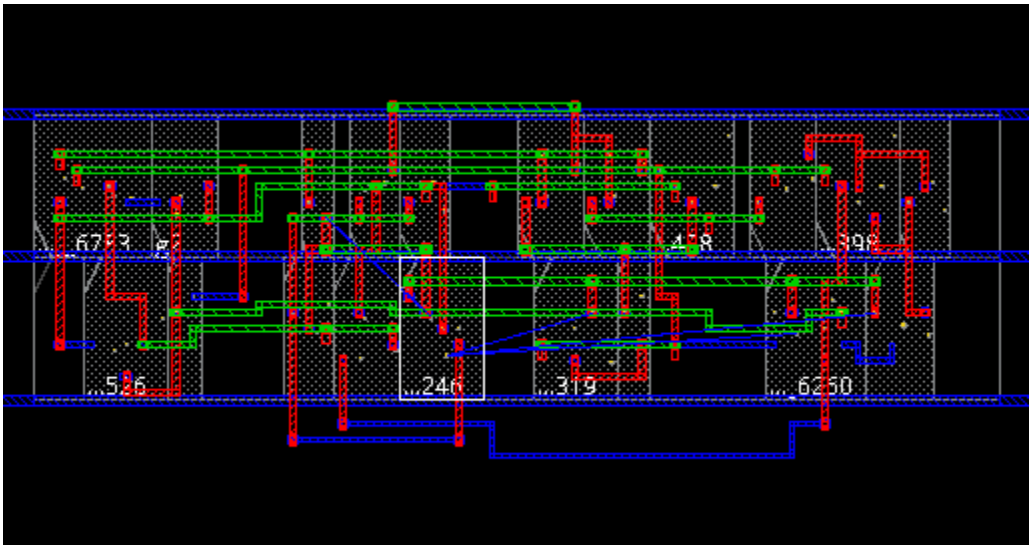


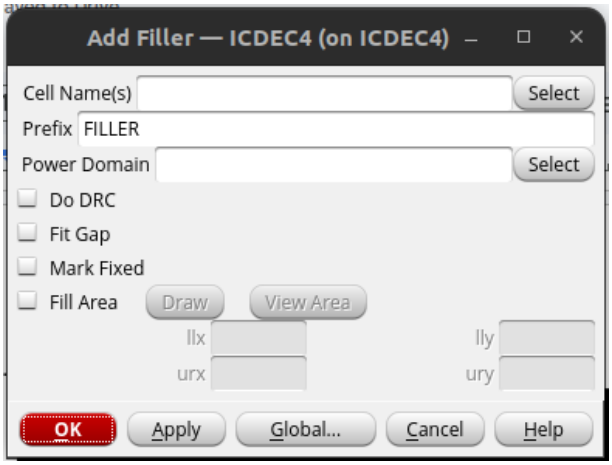
37. Pada Innovus *tool menu bar*, pilih `Route -> NanoRoute -> Route`.



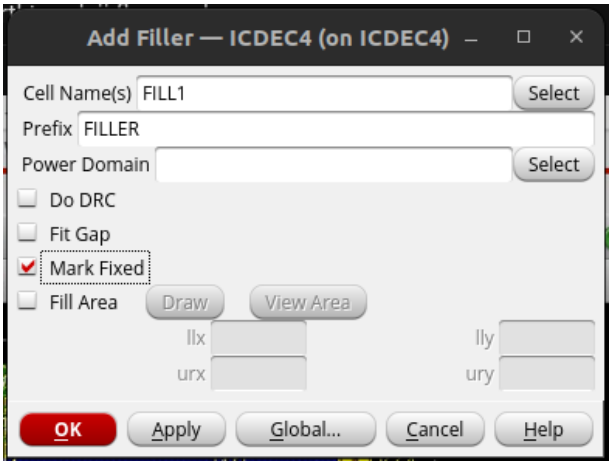
38. *Klik* OK.



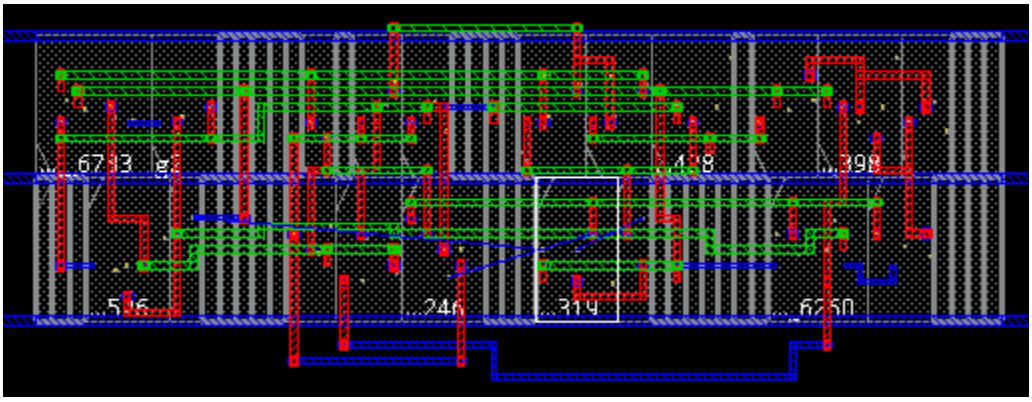
39. Selanjutnya, akan ditambahkan *Empty Filler Cells* kedalam *design*. Pilih `Place -> Physical Cell -> Add Filler` pada Innovus *tool menu bar*.



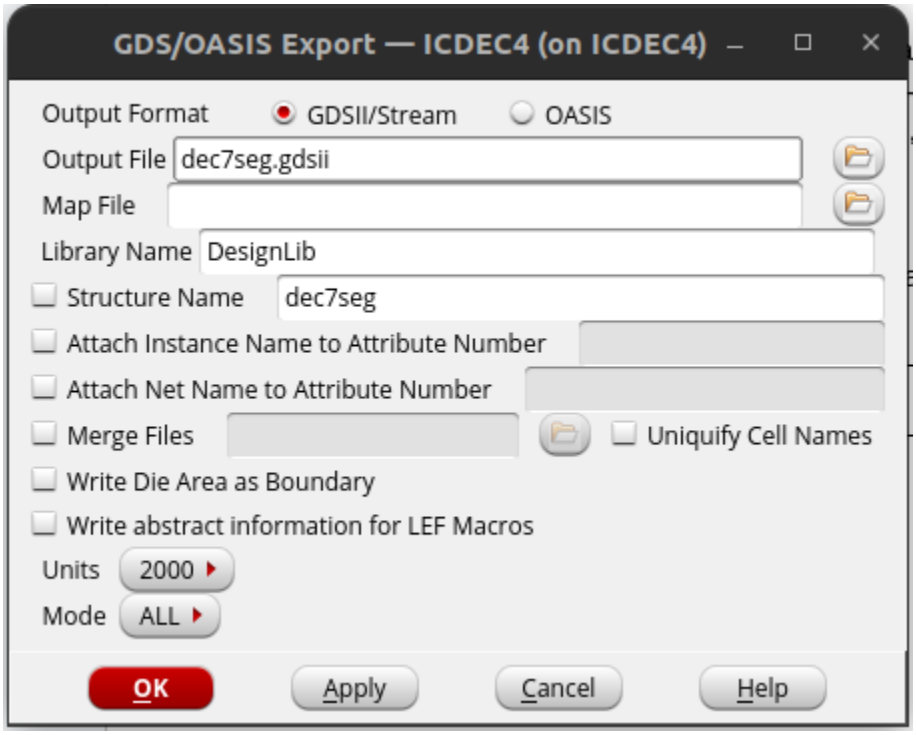

40. Pilih `FILL1` di bagian `Cell Name(s)` dan centang *box* `Mark Fixed.`



41. Jika sudah *klik* `OK`.



42. Selanjutnya, *design* dapat disimpan kedalam bentuk *file* `GDSII/OASIS`. Untuk melakukannya, pilih `File -> Save -> GDS/OASIS` pada *menu tool bar*. Pada bagian `Output Format` pilih `GDSII/Stream`, kemudian beri nama *file* pada bagian `Output File`.




43. Kemudian klik `OK`. Maka *file* `GDSII` akan tersimpan dalam direktori **`pd/`**.

44. Jika sudah, dapat keluar dari Innovus dengan *command* berikut

```
exit
```